



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Гоцкало Максим Олегович

**Детерминированные алгоритмы построения
многопроцессорного расписания
с ограничением на объем памяти**

Курсовая работа

Научный руководитель
к.ф.-м.н., с.н.с.
Балашов Василий Викторович

Москва, 2026

Аннотация

В работе рассматривается задача построения многопроцессорного расписания с минимизацией его длительности при наличии жёсткого ограничения на суммарный объём памяти, используемой для хранения промежуточных данных (буферов). Предложены два детерминированных подхода: жадный алгоритм списочного планирования с двумя эвристиками и точный метод на основе целочисленного линейного программирования (ЦЛП). Выполнено экспериментальное исследование на различных графах работ. Показано, что ЦЛП позволяет получать оптимальные расписания для малых графов (до 21 вершины), однако время его работы резко возрастает при увеличении размерности. Жадные алгоритмы работают на несколько порядков быстрее и дают решения приемлемого качества.

Содержание

Введение	5
1 Цель работы	8
2 Формальная постановка задачи	9
2.1 Модель прикладной программы	9
2.2 Модель вычислительной системы	9
2.3 Модель использования памяти	10
2.4 Модель расписания	10
2.5 Минимизируемая целевая функция	11
3 Жадный алгоритм построения многопроцессорного расписания с ограничением на память	12
3.1 Формальное описание алгоритма	12
3.2 Эвристики выбора работы	14
4 Сведение к задаче целочисленного линейного программирования	16
5 Экспериментальное исследование	21
5.1 Предмет и цели исследования	21
5.2 Классы графов работ	21
5.3 Наборы входных данных	22
5.4 Результаты экспериментов	24
5.5 Выводы	37
6 Описание программной реализации	39
6.1 Реализация жадного алгоритма	39
6.2 Реализация подхода на базе ЦЛП	42
6.3 Визуализатор временной диаграммы расписания	44
6.4 Структура входного и выходного файлов	45
Заключение	48
Список литературы	49

Введение

В современных вычислительных системах, особенно во встраиваемых приложениях, критически важным ресурсом часто является оперативная память. При выполнении набора работ (задач), связанных отношением частичного порядка (граф потока данных), каждая работа может порождать один или несколько выходных буферов, которые занимают память до момента, когда все их работы-потребители завершат выполнение. Ограниченность памяти накладывает дополнительные условия на допустимость расписания: в любой момент времени суммарный объём выделенных буферов не должен превышать объема памяти в системе. Требуется построить такое расписание (распределение работ по процессорам и порядок их выполнения), чтобы длительность расписания была минимальной.

Данная задача возникает, например, при проектировании систем цифровой обработки сигналов, где каждый вычислительный модуль (например, быстрое преобразование Фурье, фильтр нижних частот) потребляет и производит массивы данных фиксированного размера, а память является ограничивающим фактором [1]. Аналогичные проблемы характерны для научных конвейерных вычислений (scientific workflows), где промежуточные результаты могут иметь большой объём [2]. Многопроцессорное планирование с учётом ограничений памяти является NP-трудной задачей, из-за чего точные методы ограниченно масштабируются, и приходится использовать приближенные методы (например, жадные эвристики).

В работе [3] предложен жадный алгоритм для многопроцессорного списочного планирования с ограничением на долю межпроцессорных передач; в настоящей работе этот алгоритм адаптирован для учёта ограничений на память. Пример сведени задачи планирования с учетом потребления памяти к задаче ЦЛП приведен в работе [4].

На рис. 1 приведён пример графа работ. На нем прямоугольники означают работы, внутри них содержится информация вида ' $\langle \text{номер работы} \rangle$ ($\langle \text{длительность выполнения} \rangle$)'. Стрелки между прямоугольниками указывают на отношения частичного порядка между работами, на них – информация вида ' $[\langle \text{номер буфера в сквозной буферизации} \rangle: \langle \text{вес буфера} \rangle]$ '. На рис. 2 приведен пример расписания, построенного приближенным алгоритмом, а на рис. 3 – оптимального расписания. На временных диаграммах на верхних графиках прямоугольниками показываются интервалы выполнения соответствующих работ на процессорах, треугольные маркеры обозначают работы без предков,

стрелочки все так же означают зависимости между работами, а на нижних графиках показана динамика использования памяти по той же временной оси – прямоугольники означают объем используемого буфера.

Видно, что оптимальное расписание позволяет лучше использовать память и процессоры, избегая длительных простоев.

Для описанной выше задачи в работе представлены и исследованы жадные алгоритмы с двумя различными эвристиками, а также подход на основе целочисленного линейного программирования.

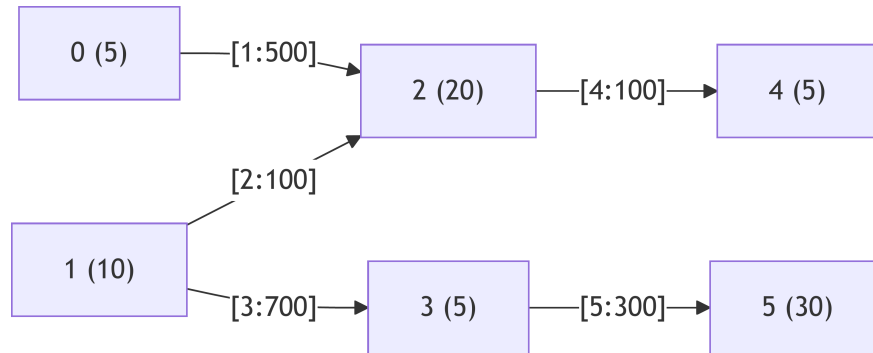


Рис. 1: Пример графа работ

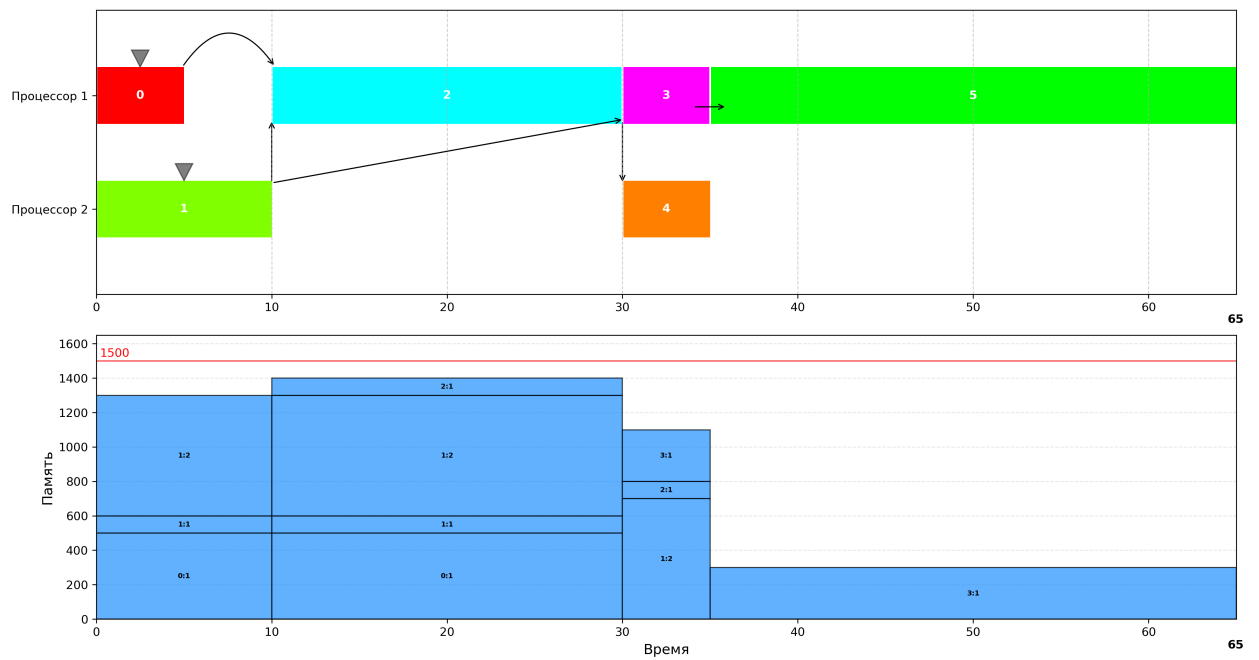


Рис. 2: Расписание, построенное приближенным алгоритмом

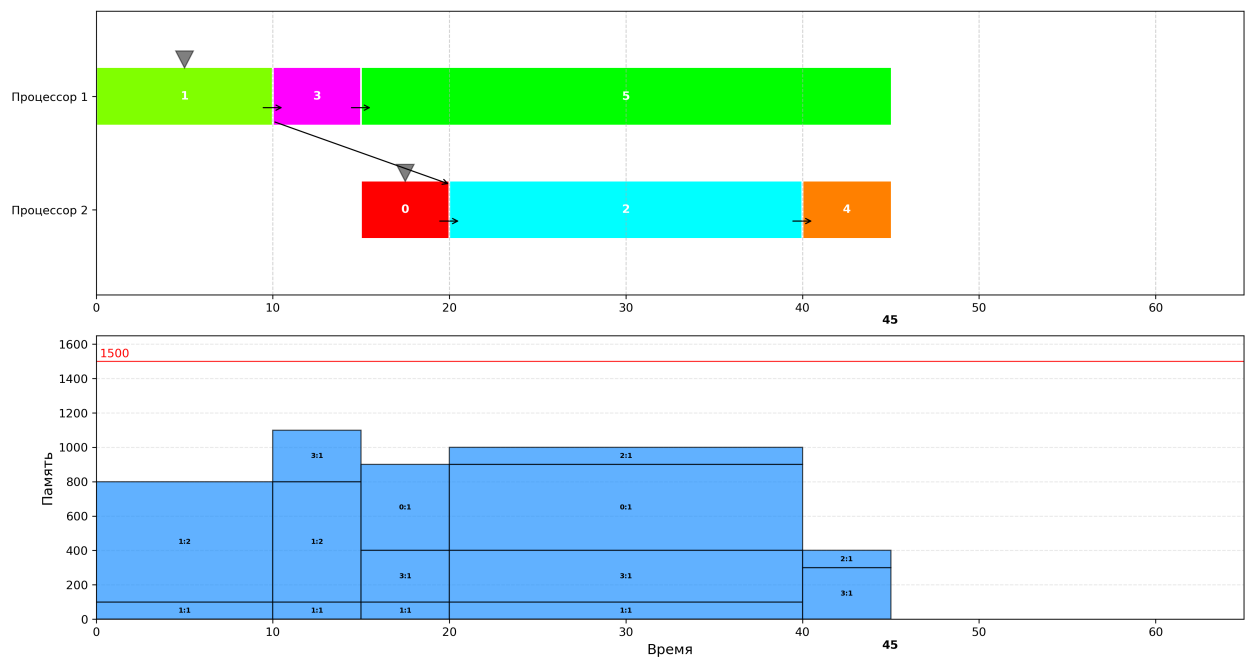


Рис. 3: Оптимальное расписание

1 Цель работы

Целью работы является разработка детерминированных алгоритмов построения многопроцессорного расписания с ограничением на объем памяти в целевой системе и минимизацией длительности расписания.

Для достижения цели необходимо:

1. разработать жадный алгоритм и эвристики в его составе;
2. выполнить сведение задачи построения расписания к задаче целочисленного линейного программирования (ЦЛП);
3. провести экспериментальное исследование жадного алгоритма и подхода на основе ЦЛП по критериям качества решения и длительности его поиска.

2 Формальная постановка задачи

2.1 Модель прикладной программы

Модель прикладной программы задается ориентированным ациклическим графом потока данных

$$G = (V, E), \quad |V| = n, \quad |E| = m.$$

Каждой вершине $v \in V$ соответствует работа, а каждой дуге $(u, v) \in E$ — зависимость по данным между работами u и v . Если $(u, v) \in E$, то работа v может начать выполняться только после завершения работы u .

Для каждой работы $v \in V$ задана длительность исполнения $d_v > 0$. Кроме того, каждая работа может формировать один или несколько выходных буферов. Обозначим множество буферов, создаваемых работой v , через

$$B(v) = \{b_{v1}, b_{v2}, \dots, b_{vk}\}.$$

Для каждого буфера $b \in B(v)$ задаются его объем памяти $r_b > 0$ и множество потребителей $C(b) \subseteq V$. Если $u \in C(b)$, то работа u использует результат, содержащийся в буфере b .

2.2 Модель вычислительной системы

Вычислительная система состоит из P одинаковых процессоров

$$SP = \{sp_1, sp_2, \dots, sp_P\}.$$

Каждый процессор в любой момент времени может выполнять не более одной работы. Прерывания работ не допускаются, миграция уже запущенной работы между процессорами невозможна.

Задержки межпроцессорной передачи данных не учитываются. Поэтому момент готовности работы к выполнению определяется завершением всех ее непосредственных предков. Ограничивающими факторами являются зависимости между работами, число доступных процессоров и общий объем памяти M , доступный для хранения активных буферов.

2.3 Модель использования памяти

Пусть работа v стартует в момент s_v и завершается в момент

$$f_v = s_v + d_v.$$

Буфер $b \in B(v)$ выделяется в момент старта работы v . В данной модели это означает предварительное резервирование памяти под результат работы: перед запуском работы система должна убедиться, что памяти достаточно для ее выходных буферов, и только после этого работа может быть начата. Освобождение буфера происходит после завершения последней из работ-потребителей его содержимого (в их вершины идут дуги, помеченные этим буфером):

$$fr_b = \max_{u \in C(b)} f_u.$$

Обозначим через $J(\tau)$ множество буферов, активных в момент времени τ . Тогда текущее использование памяти равно

$$\text{mem}(\tau) = \sum_{b \in J(\tau)} r_b.$$

Пиковое использование памяти на расписании HP определяется как

$$\text{peak}(HP) = \max_{0 \leq \tau \leq \text{makespan}} (\text{mem}(\tau)).$$

Расписание считается допустимым по памяти, если выполнено условие

$$\text{peak}(HP) \leq M.$$

2.4 Модель расписания

Расписание HP задает для каждой работы $v \in V$:

1. номер процессора $p(v) \in \{1, \dots, P\}$;
2. момент старта s_v ;
3. момент завершения $f_v = s_v + d_v$.

Расписание является корректным, если выполняются следующие ограничения:

1. каждая работа назначена ровно одному процессору;
2. на каждом процессоре интервалы выполнения назначенных ему работ не пересекаются;
3. для каждой дуги $(u, v) \in E$ выполнено неравенство $f_u \leq s_v$;
4. для всего расписания выполнено ограничение по памяти $\text{peak}(HP) \leq M$.

2.5 Минимизируемая целевая функция

Минимизируемым критерием является длительность расписания, то есть момент завершения последней работы:

$$C_{\max}(HP) = \max_{v \in V} f_v.$$

Требуется найти такое допустимое расписание HP^* , что

$$C_{\max}(HP^*) = \min_{HP \in \mathcal{H}_{\text{adm}}} C_{\max}(HP),$$

где $\mathcal{H}_{\text{adm}}$ — множество всех корректных расписаний.

Похожие задачи рассматривались в работах [5], [6] и [7], а в работе [8] доказывается, что подобная задача является NP-трудной.

3 Жадный алгоритм построения многопроцессорного расписания с ограничением на память

Жадный алгоритм (ЖА) нужен для быстрого построения корректного расписания на основании некоторой эвристики. При этом не гарантируется оптимальность построенного расписания, но в основе алгоритма лежит локальная стратегия, которая на каждом шаге алгоритма выглядит хорошей с точки зрения минимизации целевой функции. Жадный алгоритм является конструктивным и последовательно достраивает расписание - на каждом шаге алгоритма в соответствии с некоторым критерием выбирается новая работа и добавляется в это расписание. Такой ЖА относится к алгоритмам списочного планирования. Жадные алгоритмы для похожих задач описаны в работах [3] и [9].

3.1 Формальное описание алгоритма

Алгоритм работает следующим образом.

Шаг 1. Инициализация.

- Множество ещё не назначенных работ $\mathcal{U} = V$, множество назначенных работ $\mathcal{U}' = \emptyset$.
- Множество готовых работ $\mathcal{R} = \{i \in V \mid \text{у } i \text{ нет предшественников}\}$.
- Для каждого процессора $p = 1, \dots, P$ будет храниться набор Bus_p интервалов занятости $[t_{\text{start}}, t_{\text{end}})$, изначально все $Bus_p = \emptyset$.
- Для каждого буфера (i, j) вводится интервал активности $Act_{i,j} = [alloc_{i,j}, free_{i,j})$, где $alloc_{i,j}$ будет означать время, когда память была выделена под буфер, а $free_{i,j}$ – когда эта память была освобождена. Изначально все $alloc_{i,j} = free_{i,j} = 0$ (буферы не используются).

Шаг 2. Для каждой работы $i \in \mathcal{R}$ и для каждого процессора p :

1. Найти наиболее раннее время t такое, что:

- процессор p свободен на интервале $[t, t + d_i)$ (нет пересечений с интервалами из Bus_p);

- для всех предшественников j работы i выполнено $s_j + d_j \leq t$;
 - Для всех моментов $\tau \in [t, t + d_i)$ суммарный объём буферов, у которых еще не все потребители поставлены в расписание, а также буферов (i', j') , таких, что $\tau \in Act_{i,j}$, не превышает $(M - sum_r_i)$, где sum_r_i – суммарный объем всех буферов работы i .
2. Если такое t существует, запомнить для пары (i, p) возможное время старта $t_{i,p}$.
 3. Среди всех пар (i, p) , для которых найдено допустимое $t_{i,p}$, выбрать одну работу i^* и процессор p^* согласно выбранной эвристике (см. подраздел 3.2). Если ни одной допустимой пары нет, СТОП (неуспех) – не удалось построить корректное расписание, либо задача в принципе не имеет решения.
 4. Зафиксировать работу i^* на процессоре p^* со стартом $t = t_{i^*,p^*}$. Обновить расписание: добавить интервал занятости $[t, t + d_{i^*})$ для процессора p^* в набор Bus_{p^*} , зафиксировать момент старта $s_{i^*} = t$ и завершения $c_{i^*} = t + d_{i^*}$. Обновить состояние памяти: положить $Act_{i,j} = [t, t + d_{i^*})$, а для всех буферов (i', j') , таких, что работа i^* использует этот буфер, положить $free_{i',p'} = \max(free_{i',p'}, t + d_{i^*})$.

Шаг 3. Удалить i^* из $\mathcal{U}$ и $\mathcal{R}$, добавить i^* в $\mathcal{U}'$. Добавить в $\mathcal{R}$ все работы, у которых работа i^* была последним неназначенным предком. Если $\mathcal{U} \neq \emptyset$, перейти к шагу 2.

Шаг 4. СТОП.

В результате получаем полное расписание — для каждой работы определены процессор и время старта.

Блок-схема работы ЖА представлена на рисунке 4.



Рис. 4: Блок-схема ЖА

3.2 Эвристики выбора работы

Ключевым элементом жадного алгоритма является правило выбора работы из множества готовых. В данной работе рассматриваются две эвристики: **EFT** (Earliest Finish Time) и **STS** (Smallest Time-Space).

Эвристика EFT

Эвристика EFT ориентирована на минимизацию времени завершения работы. После того как для каждой готовой работы i и каждого процессора p определено наиболее раннее допустимое время старта $t_{i,p}$, вычисляется время завершения:

$$f_{i,p} = t_{i,p} + d_i.$$

Затем выбирается пара (i^*, p^*) , дающая наименьшее $f_{i,p}$. В случае равенства предпочтение отдается работе с наименьшим номером (или процессору с наименьшим номером, если несколько процессоров дают одинаковое время завершения для одной работы). Эта эвристика отличается простотой вычисления, но никак не учитывает использование памяти выбираемой работой.

Эвристика STS

Эвристика STS введена для учёта влияния работы на использование памяти в будущем. Оценка работы складывается из двух компонент.

Первая компонента учитывает собственные буферы работы:

$$S_i^{(1)} = d_i \cdot \sum_{j=1}^{n_i} r_{ij}.$$

То есть произведение длительности работы на суммарный объём создаваемых ею буферов.

Вторая компонента учитывает «нагрузку» на память, создаваемую потомками работы. Для каждого буфера j работы i определим множество его потребителей $Cons(i, j) = \{k \mid (i, k, j) \in E\}$. Пусть $d_{total}(i, j)$ — суммарная длительность всех работ-потребителей этого буфера (каждый потребитель учитывается один раз, даже если он использует несколько буферов). Тогда

$$S_i^{(2)} = \sum_{j=1}^{n_i} \left(r_{ij} \cdot \sum_{k \in Cons(i, j)} d_k \right).$$

Общая оценка работы i равна

$$\phi(i) = S_i^{(1)} + S_i^{(2)}.$$

Алгоритм выбирает работу с наименьшим значением $\phi(i)$. Если несколько работ имеют одинаковую оценку, то среди них выбирается работа с наименьшим временем завершения (т.е. срабатывает правило EFT). Идея эвристики STS заключается в том, что работа, которая создаёт большие буферы и эти буферы нужны долго выполняющимся потомкам, потребляет память на длительное время, поэтому её выгодно выполнить позже (чтобы не блокировать память). Соответственно, на ранних шагах следует выбирать работы с малым $\phi(i)$.

4 Сведение к задаче целочисленного линейного программирования

Базовое описание метода линейного программирования содержится в работе [10]. За основу предлагаемого в данной работе подхода взято сведение однопроцессорного варианта задачи к ЦЛП из работы [4].

Входные данные

Каждая работа i ($i = 1, \dots, n$) характеризуется длительностью $d_i > 0$ и набором буферов, которые она создаёт:

$$V = \{(i, n_i, r_{i1}, r_{i2}, \dots, r_{in_i})\}_{i=1}^n,$$

где n_i — количество буферов работы i , а $r_{ij} > 0$ — объём памяти, занимаемый j -м буфером. Дуги графа задают передачу данных:

$$E = \{(i, k, j)\}, \quad j \in [1, n_i],$$

т.е. дуга (i, k, j) означает, что j -й буфер работы i используется работой k (является её входным буфером). Все времена d_i целые, допустимый объём памяти равен M , число процессоров P .

Переменные

- s_i — время старта работы i (целочисленная переменная).
- m_{ij} — бинарная переменная, $m_{ij} = 1$ если $s_i \leq s_j$, иначе 0 ($i, j = 1, \dots, n$).
- p_{ri} — бинарная переменная, $p_{ri} = 1$ если работа i назначена на процессор r ($r = 1, \dots, P$).
- t_{ij} — бинарная переменная, $t_{ij} = 1$ если работы i и j выполняются на одном процессоре.
- l — целочисленная переменная, общая длительность расписания (makespan).

- Для каждого буфера (i, j) ($j = 1, \dots, n_i$) вводятся целочисленные переменные:

$$\text{start}_{ij}, \quad \text{max_end}_{ij}.$$

start_{ij} совпадает со стартом работы i , а max_end_{ij} — максимальное время завершения среди всех потребителей этого буфера.

- Для каждой пары буферов (i, j) и (p, q) вводятся бинарные переменные:

$$z_{ij}^{pq}, \quad z_{ij}^{pq}, \quad a_{ij}^{pq}, \quad aa_{ij}^{pq}.$$

Они служат для моделирования пересечений интервалов активности буферов.

Вспомогательная константа

$$D = \sum_{i=1}^n d_i,$$

которая заведомо больше любой возможной длительности расписания.

Ограничения, связывающие m_{ij} и s_i

Для всех i, j (при $i = j$ полагаем $m_{ii} = 1$):

$$m_{ij} \geq \frac{s_j - s_i}{D}, \quad \forall i \neq j, \quad (1)$$

$$1 - m_{ij} \geq \frac{s_i - s_j + 1}{D}, \quad \forall i \neq j, \quad (2)$$

$$m_{ii} = 1. \quad (3)$$

Неравенство (1) гарантирует, что если $s_i \leq s_j$, то $m_{ij} = 1$ (при выполнении (2) обратное).

Константа D выбрана так, что $|s_j - s_i| < D$.

Назначение на процессоры

Для всех i и r :

$$\sum_{r=1}^P p_{ri} = 1, \quad (4)$$

$$p_{ri} \in \{0, 1\}.$$

Каждая работа назначается ровно на один процессор.

Ограничения на переменные t_{ij} (один процессор)

Для всех i, j, k ($i \neq j$) и всех r :

$$t_{ij} \geq p_{ri} + p_{rj} - 1, \quad (5)$$

$$t_{ij} \leq p_{ri} - p_{rj} + 1, \quad (6)$$

$$t_{ij} \in \{0, 1\}.$$

Ограничения (5) и (6) означают, что $t_{ij} = 1$ тогда и только тогда, когда $p_{ri} = p_{rj} = 1$ для некоторого r .

Ограничения частичного порядка (из графа работ)

Для каждой дуги $(i, k, j) \in E$ (работа k потребляет j -й буфер работы i):

$$s_k \geq s_i + d_i. \quad (7)$$

Определение start_{ij} и max_end_{ij}

Для каждого буфера (i, j) :

$$\text{start}_{ij} = s_i, \quad (8)$$

$$\text{max_end}_{ij} \geq s_k + d_k \quad \text{для всех } k \text{ таких, что } (i, k, j) \in E. \quad (9)$$

Таким образом, max_end_{ij} — максимум моментов завершения всех потребителей буфера.

Взаимное исключение на одном процессоре

Для всех $i \neq j$:

$$s_i - s_j + D m_{ij} - D t_{ij} \geq -D + d_j, \quad (10)$$

$$s_j - s_i - D m_{ij} - D t_{ij} \geq -2D + d_i. \quad (11)$$

Если $t_{ij} = 1$ (работы на одном процессоре), то эти ограничения вынуждают $s_j \geq s_i + d_i$ при $m_{ij} = 1$ и $s_i \geq s_j + d_j$ при $m_{ij} = 0$. При $t_{ij} = 0$ ограничения выполняются автоматически.

Учёт пересечений буферов (для ограничения памяти)

Для каждой пары буферов (i, j) и (p, q) введём вспомогательные бинарные переменные z_{ij}^{pq} , zz_{ij}^{pq} , a_{ij}^{pq} , aa_{ij}^{pq} . Они связаны следующими линейными ограничениями:

$$z_{ij}^{pq} \leq \frac{\text{start}_{pq} - \text{start}_{ij}}{D + 1} + \frac{D}{D + 1}, \quad (12)$$

$$a_{ij}^{pq} \geq \frac{\text{max_end}_{pq} - \text{start}_{ij}}{D + 1} - z_{ij}^{pq}, \quad (13)$$

$$zz_{ij}^{pq} \leq \frac{\text{start}_{pq} - \text{max_end}_{ij}}{D + 1} + \frac{D}{D + 1}, \quad (14)$$

$$aa_{ij}^{pq} \geq \frac{\text{max_end}_{pq} - \text{max_end}_{ij}}{D + 1} - zz_{ij}^{pq}. \quad (15)$$

Пояснение:

- (12) гарантирует, что если $\text{start}_{pq} \leq \text{start}_{ij}$, то $z_{ij}^{pq} = 0$; в противном случае z_{ij}^{pq} может быть 1.
- (13) совместно с (12) обеспечивает $a_{ij}^{pq} = 1$, когда интервал активности (p, q) покрывает момент start_{ij} .
- (14) аналогично (12), но для момента max_end_{ij} .
- (15) аналогично (13), но для момента max_end_{ij} .

Интервал активности буфера (p, q) — это $[\text{start}_{pq}, \text{max_end}_{pq})$.

Ограничение на суммарную память

Для каждого буфера (i, j) суммарный объём памяти, занятый в моменты start_{ij} и max_end_{ij} , не должен превышать M :

$$\sum_{(p,q)} r_{pq} a_{ij}^{pq} \leq M, \quad (16)$$

$$\sum_{(p,q)} r_{pq} aa_{ij}^{pq} \leq M, \quad (17)$$

где сумма берётся по всем буферам, присутствующим в графе. Эти ограничения гарантируют, что пиковое потребление памяти в любой момент (все точки вида start_{ij} или max_end_{ij}) не превышает M .

Переменная для целевой функции

Верхняя оценка длительности расписания (при минимизации становится равна этой длительности):

$$l \geq s_i + d_i \quad \forall i. \quad (18)$$

Итоговая задача

Минимизировать целевую функцию, равную l , при ограничениях (1)–(18). Переменные $s_i, \text{start}_{ij}, \text{max_end}_{ij}, l \in \mathbb{Z}_{\geq 0}, m_{ij}, p_{ri}, t_{ij}, z_{ij}^{pq}, zz_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq} \in \{0, 1\}$.

Число переменных и ограничений

Пусть n — число работ, n_i — число буферов работы i , $B = \sum_i n_i$ — общее число буферов, P — число процессоров. Тогда:

- m_{ij} : n^2 бинарных.
- s_i : n целых.
- p_{ri} : Pn бинарных.
- t_{ij} : n^2 бинарных.
- $\text{start}_{ij}, \text{max_end}_{ij}$: $2B$ целых.
- $z_{ij}^{pq}, zz_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq}$: $4B^2$ бинарных.
- l : одна целая.

Количество ограничений составляет $O(n^2 + Pn + B^2)$.

5 Экспериментальное исследование

5.1 Предмет и цели исследования

Предметом исследования являются предложенные подходы к решению задачи: жадный алгоритм с эвристиками EFT и STS; подход на основе ЦЛП. Целями исследования является сравнительный анализ подходов по следующим критериям:

- качество получаемого решения (т.е. значение целевой функции на расписании);
- масштабируемость по времени поиска решения при увеличении размерности входных данных.

5.2 Классы графов работ

Используются три класса ориентированных ациклических графов:

Слоистые графы

Слоистые графы (рис. 5) характерны для многоэтапных вычислений (цифровая обработка сигналов, научные расчёты). Вершины разбиты на последовательные слои, рёбра в основном соединяют вершины соседних слоёв, но допускаются «перескоки» через несколько слоёв.

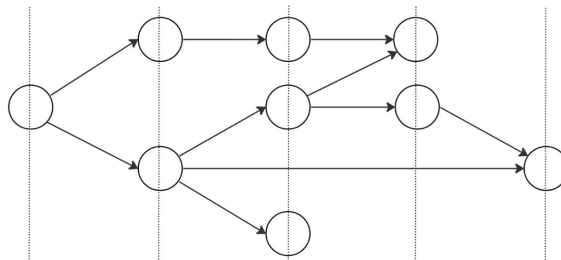


Рис. 5: Пример слоистого графа

Случайные графы

Случайные графы (рис. 6) генерируются с ограничениями: у каждой вершины не более двух предков и не более шести потомков. Такие графы встречаются, например, при реализации LU- и QR-разложений.

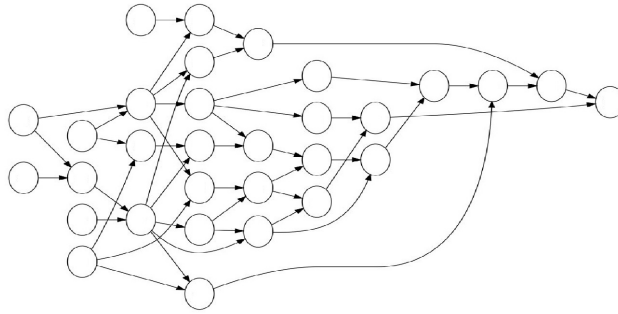


Рис. 6: Пример случайного графа

Треугольные графы

Треугольные графы (рис. 7) — частный случай слоистых графов, моделирующий метод Гаусса–Жордана. Они обладают регулярной структурой: каждый слой содержит на одну вершину меньше предыдущего, рёбра соединяют только соседние слои. Используются только для больших размеров задач.

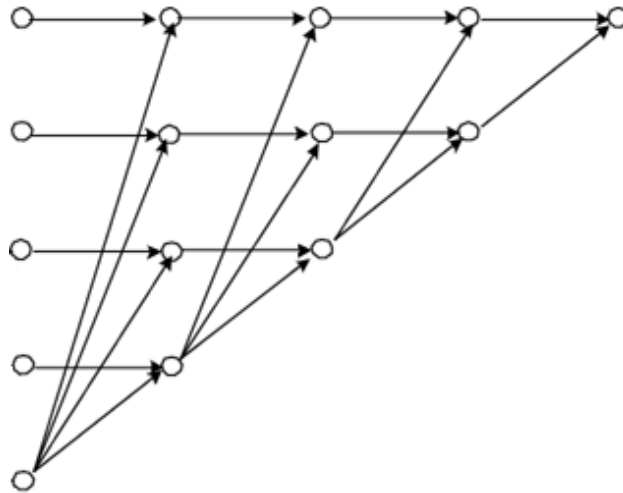


Рис. 7: Пример треугольного графа

5.3 Наборы входных данных

Для каждого из двух исследований (качество и масштабируемость) используются отдельные наборы графов, различающиеся размерностью. Все графы имеют следующие общие параметры:

- Длительность работы d_i — целое число, равномерно распределённое в диапазоне $[10, 100]$.
- Вес каждого буфера — целое число, равномерно распределённое в диапазоне $[1, 100]$.
- Для каждой работы используются два варианта формирования буферов:
 1. **Один буфер:** все исходящие рёбра объединены в один буфер указанного веса.
 2. **Корневое равномерное распределение:** количество буферов равно $\lceil \sqrt{\text{outdeg}} \rceil$, рёбра распределяются по буферам максимально равномерно.

Малые графы

- **Слоистые графы:** все размеры $n = 10, 11, \dots, 21$ (по одному графу на размер).
- **Случайные графы:** все размеры $n = 10, 11, \dots, 21$ (по одному графу на размер).

Большие графы

- **Слоистые графы:** по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$.
- **Случайные графы:** по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$.
- **Треугольные графы:**
 - Для оценки качества: по одному графу для каждого размера $n \in \{55, 210, 496, 990\}$
 - Для оценки масштабируемости: расширенная сетка размеров - по одному графу для каждого размера $n \in \{55, 105, 153, 210, 300, 496, 741, 990\}$

Сетка параметров (P, M)

Для каждого графа предварительно определяются:

- $P_{\max}$ — максимальное число процессоров, которое может эффективно использовать жадный алгоритм на данном графе при заведомо избыточной памяти.
- $M_{\max}$ — максимальная пиковая память, возникающая при выполнении расписания с заведомо избыточным числом процессоров.

Число процессоров P :

- Если $P_{\max} \geq 50$, то $P \in \{4, \lfloor (4 + P_{\max})/2 \rfloor, P_{\max}\}$.
- Если $P_{\max} < 50$, то $P \in \{2, \lfloor (2 + P_{\max})/2 \rfloor, P_{\max}\}$.

Лимит памяти M :

- Для малых графов ($n \leq 21$): $M_{\min} = 1$, поиск ведётся с шагом 1 до первого допустимого расписания.
- Для больших графов ($n \geq 55$): $M_{\min} = \lfloor M_{\max}/10 \rfloor$, шаг поиска $\Delta M = \lfloor M_{\max}/100 \rfloor$.
- В обоих случаях используются три уровня: $M \in \{M_{\min}, \lfloor (M_{\min} + M_{\max})/2 \rfloor, M_{\max}\}$.

Таким образом, для каждого исходного графа формируется 9 вариантов входных данных (3×3 комбинации P и M), на которых проводятся запуски.

5.4 Результаты экспериментов

Эксперименты проводились на компьютере со следующими характеристиками:

- ЦП: Apple M1
- Количество ядер: 8;
- Объём ОЗУ: 16 Гбайт;
- ОС: macOS Sonoma 14.5.

Выбор решателя и схема параллельного запуска В качестве решателя задач целочисленного линейного программирования используется SCIP. Выбор обоснован в работе [4], где проведено сравнительное исследование производительности свободно доступных решателей (GLPK, CBC, SCIP) на графах рассматриваемых классов; SCIP показал наилучшие результаты по времени поиска оптимального решения.

Из-за значительной зависимости времени работы решателя от упорядочения переменных и ограничений, а также от добавления транзитивных рёбер, применяется метод «параллельного запуска с остановкой по первому успеху». Для каждого входного графа формируются 8 вариантов .lp-файлов, соответствующих комбинациям:

- 4 упорядочения вершин и рёбер: `default`, `up_right`, `down_left`, `tiers`;
- 2 варианта графа: с добавленными транзитивными ограничениями (флаг `-tr`) и без них.

(Упорядочение `reverse_tiers` не используется из-за ограниченности числа доступных процессорных ядер.)

Затем скрипт `run_scip2.sh` одновременно запускает 8 экземпляров SCIP по одному на каждый вариант. Как только хотя бы один экземпляр завершается с нахождением оптимального решения (статус `optimal solution found`), все остальные процессы принудительно завершаются. Это позволяет избежать неоправданного ожидания на «медленных» упорядочениях и эффективно использовать доступные вычислительные ресурсы. Лимит времени на каждый экземпляр задаётся в файле `only_time.set` (в экспериментах использовалось 3600 секунд).

Малые графы

Исследование масштабируемости

При исследовании масштабируемости целью является построить график зависимости времени работы алгоритмов от размера графов. Для этого необходимо зафиксировать остальные параметры графов, такие как класс, неоднородность буферов, значения P и M . Но при этом P и M различаются у каждого графа, поэтому решено было фиксировать порядок этих параметров: так как их всегда по 3 варианта для каждого графа - малое, среднее и большое - то можно для всех рассматриваемых графов выбрать один из 9 типов этих параметров, например, малое P средний M . Также было решено добавить график целевой функции для дополнительного сравнения. На рисунке 8 видно, что ЛП завершается на всех размерах графов, так же, как и ЖА с EFT. На рисунке 9 видно, что ЛП не завершается на графах начиная с 16 вершин при таких параметрах P и M , при этом уже на 14-15 вершинах время работы около 1000 секунд, то есть уже близко к установленному лимиту времени выполнения. На всех графиках видно, что ЛП работает заметно дольше обоих ЖА, но при этом везде выдает расписание не хуже, а зачастую лучше по `makespan`.

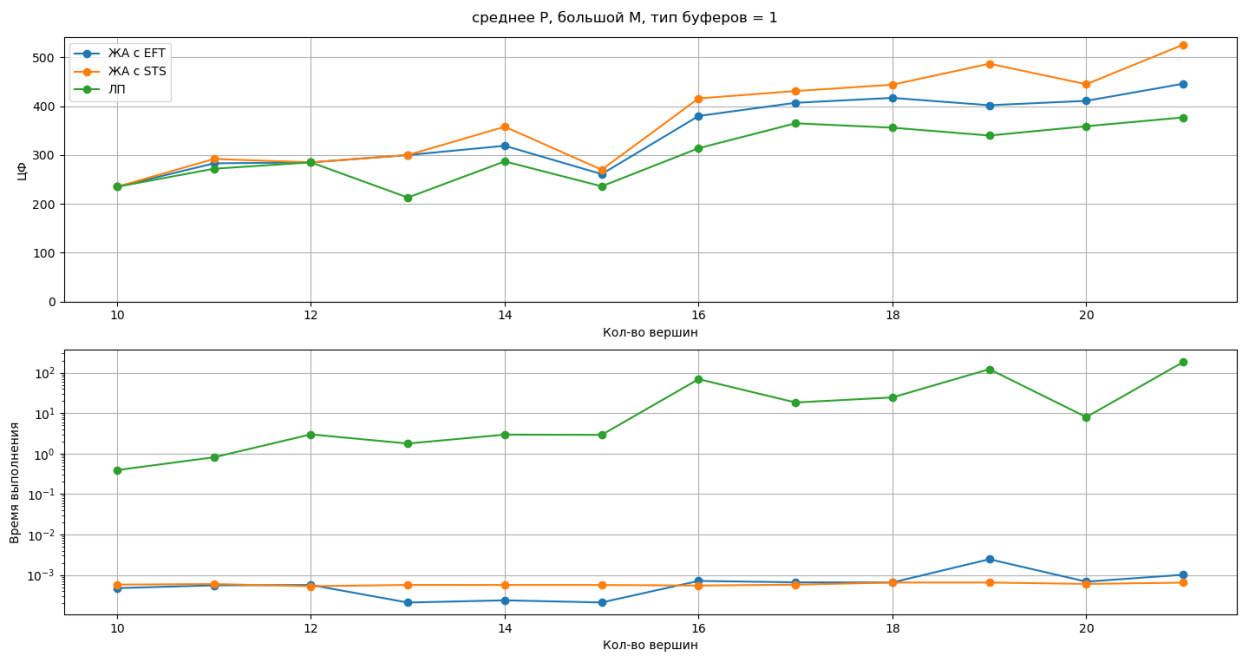


Рис. 8: Все алгоритмы успешно завершаются

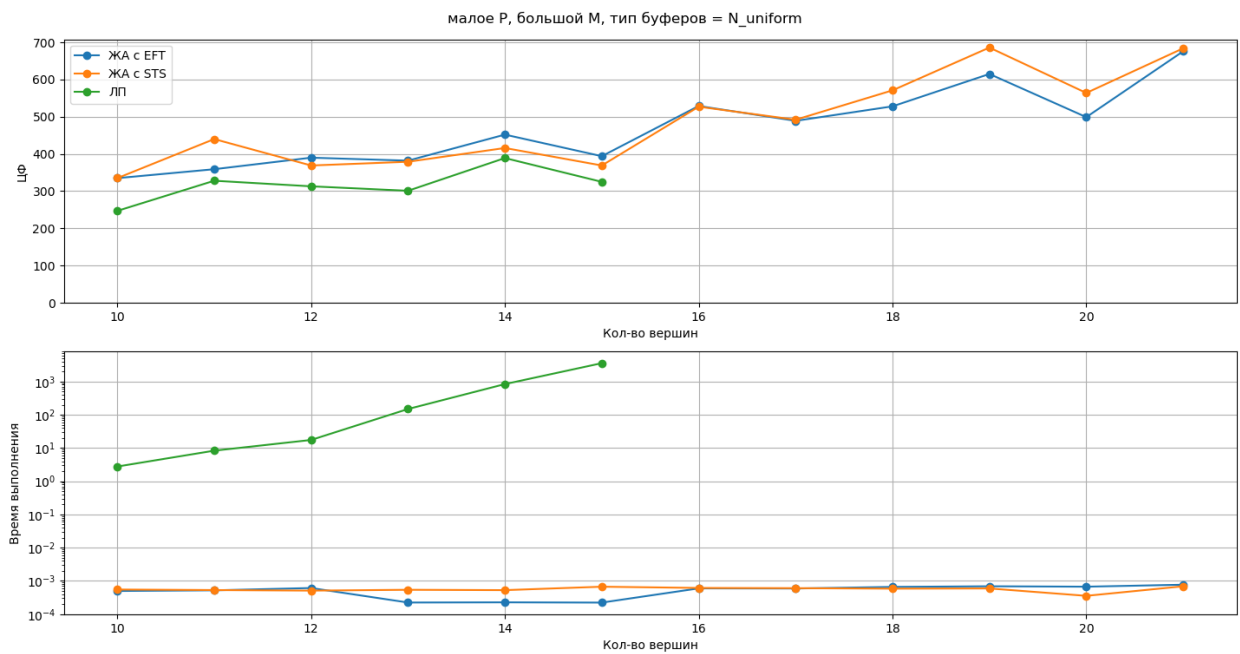


Рис. 9: ЛП сильно замедляется

Исследование качества решения

При исследовании качества решения необходимо сравнить значения целевой функции разных алгоритмов на одних и тех же графах на одних и тех же параметрах. На этот раз на одной диаграмме решено было зафиксировать класс графа, его размер и тип буферов, и строить столбчатые диаграммы значения ЦФ для каждой пары параметров P и M . Снизу дополнительно добавили график времени выполнения. На рисунке 10 видно, как все алгоритмы завершаются на всех параметрах, можно заметить, что ЛП всегда не хуже обоих ЖА, что есть параметры, при которых один ЖА может быть лучше другого и наоборот, а также уже можно наблюдать снижение значения ЦФ при росте числа процессоров и лимита памяти. На рисунке 11 видно, как на случайном графе размера 17 ЛП еще работает на всех параметрах, в отличие от ЖА с STS, но время выполнения уже близко к лимиту. И на рисунке 12 видно, что уже на 18 вершинах случайного графа ЛП не завершается на 2 процессорах. В целом на всех диаграммах можно заметить, что ЛП всегда не хуже обоих ЖА, при этом работает сильно дольше них, также заметно, что ЖА с STS не всегда завершается на параметрах, которые были подобраны для ЖА с EFT.

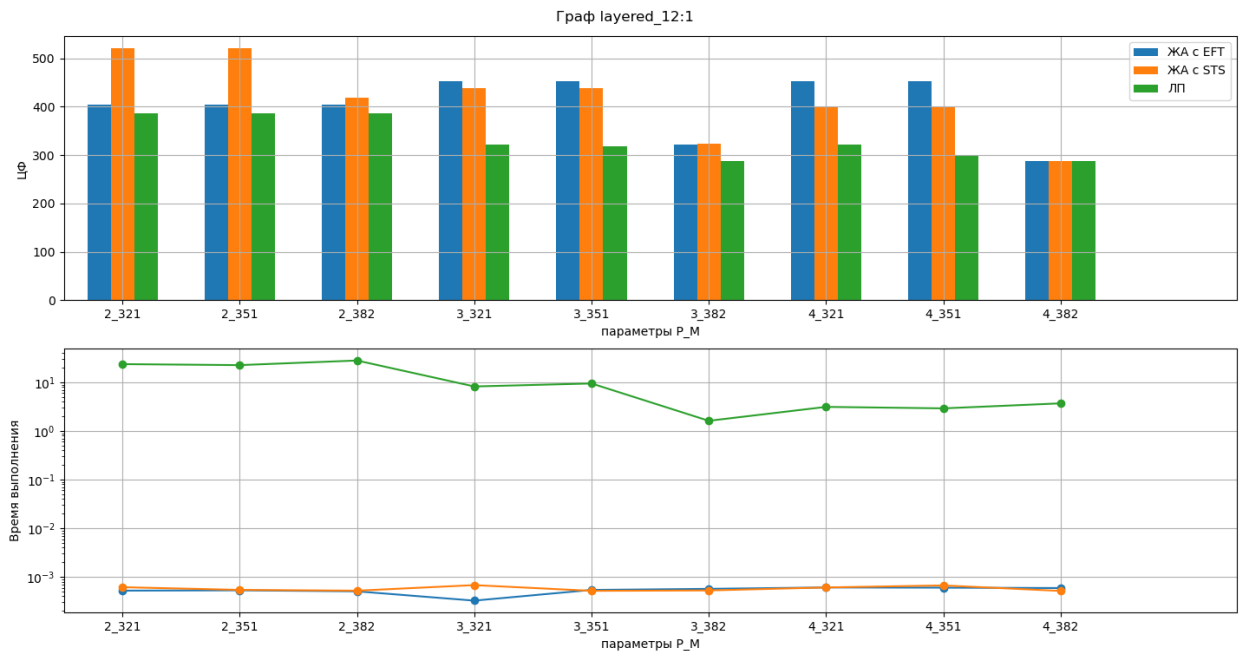


Рис. 10: Все алгоритмы завершаются

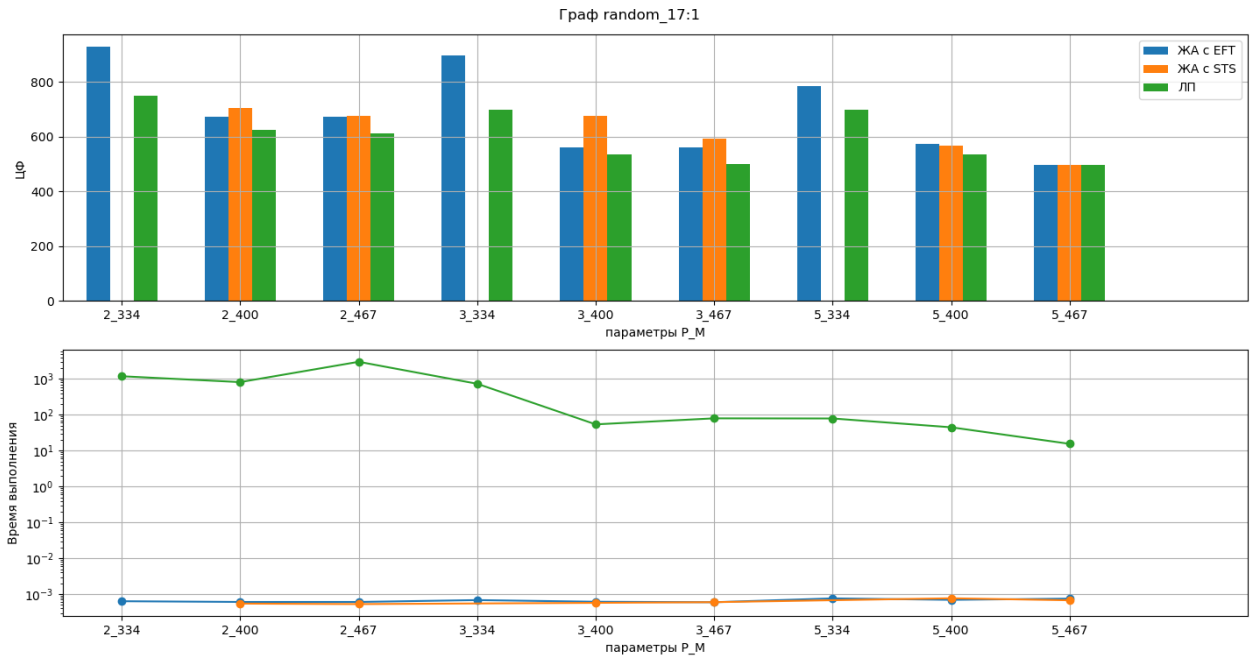


Рис. 11: Не всегда завершается ЖА с STS и замедление ЛП

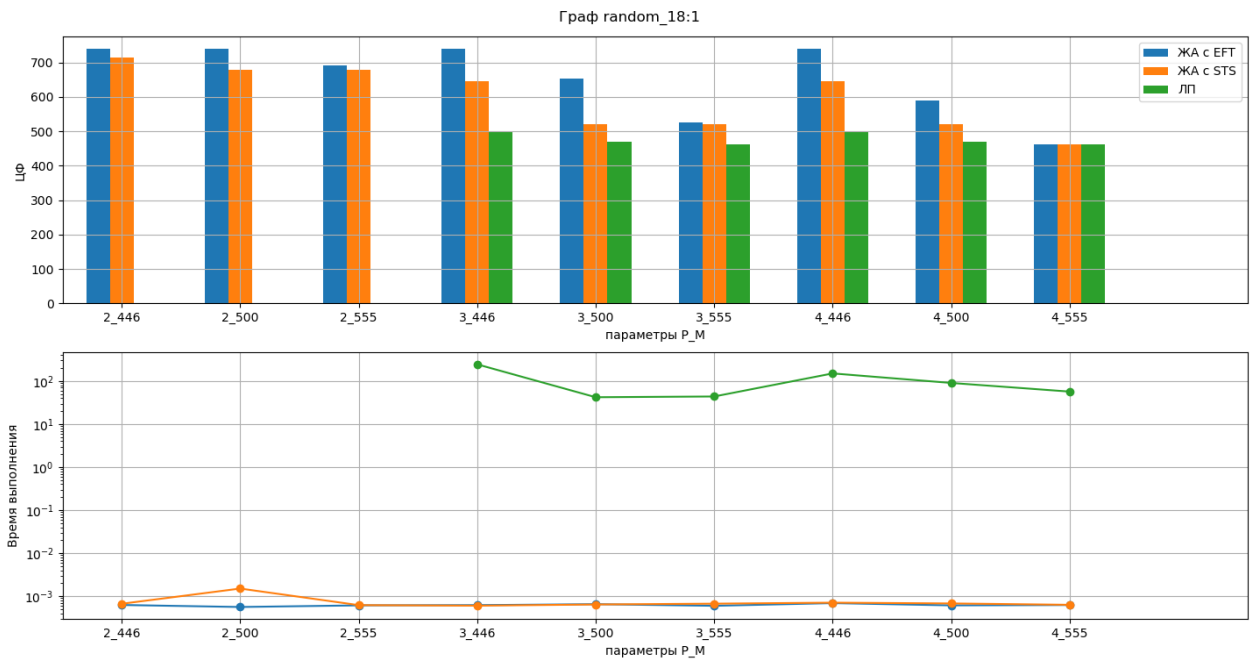


Рис. 12: Сильное замедление ЛП

На указанных выше рисунках также можно заметить, как оптимальное значение ЦФ, получаемое с помощью ЛП, почти всегда снижается при увеличении лимита па-

мента при одном и том же числе процессоров. Аналогичное снижение можно наблюдать при увеличении числа процессоров при неизменной памяти. Это можно заметить и для обоих ЖА, хотя для них такое поведение не гарантируется. При этом разница между ЛП и ЖА заметнее всего снижается при увеличении лимита памяти, при максимальных значениях Р и М результаты работы всех алгоритмов и вовсе совпадают. При этом на время работы ЛП оказывает существенное влияние только число процессоров (при $P = 2$ алгоритм даже не всегда завершается).

Характерные случаи

На рисунках 13- 14 видно, как ЛП может быть лучше ЖА. Видно, что ЖА использует всего 3 процессора из 6 доступных, так как сразу ставит параллельно выполняться 3 корневых работы, а далее ему не хватает памяти для размещения готовых работ на новые процессоры, тогда как ЛП находит способ сначала выполнить работы с большими размерами буферов, а далее ставить выполняться параллельно больше число работ. Граф - layered_20_N_uni_6_464.

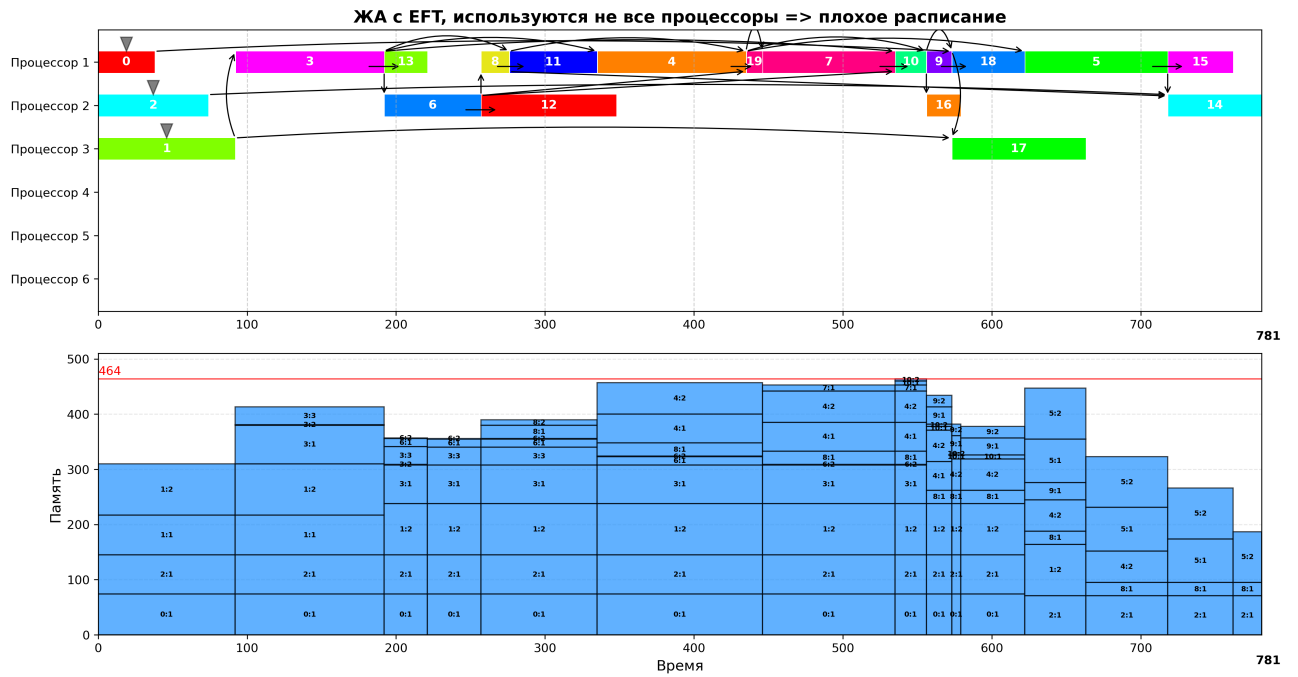


Рис. 13: Неэффективное использование процессоров

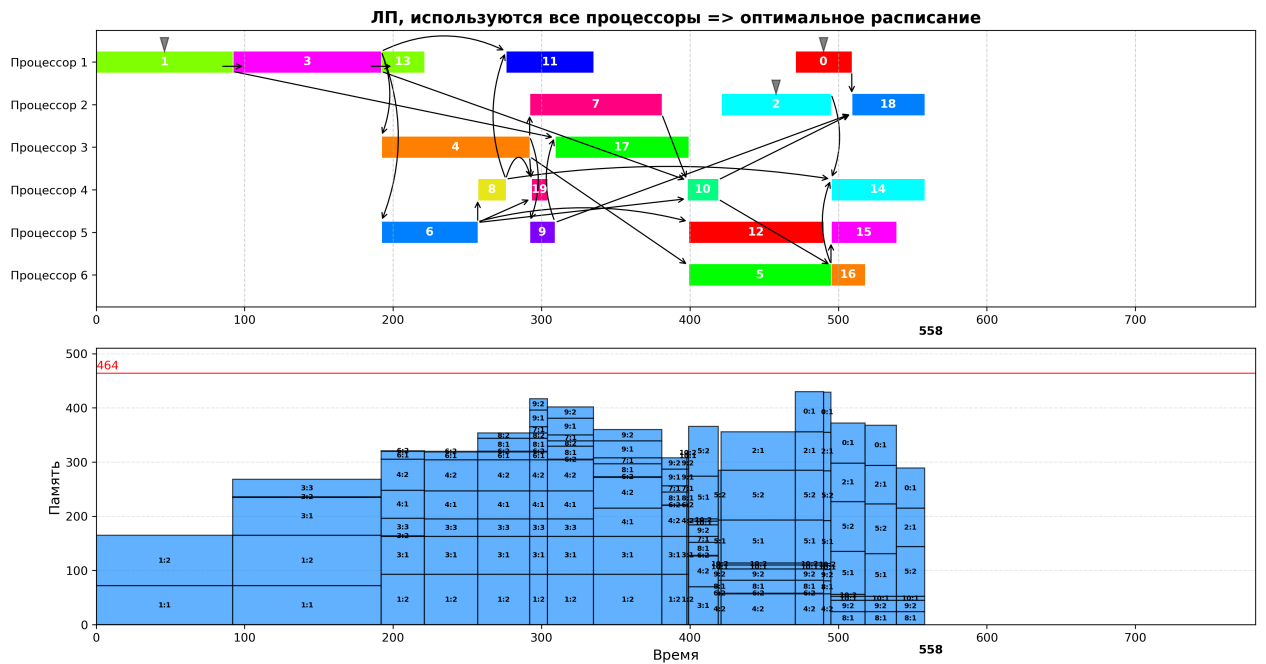


Рис. 14: Использование большего числа процессоров

Для обобщения результатов для каждой комбинации (тип графа, размер, вариант

буферизации, Р, М) вычисляется относительное отклонение makespan жадных алгоритмов от оптимального значения, полученного ЦЛП:

$$\delta_{EFT} = \frac{\text{makespan}_{EFT} - \text{makespan}_{opt}}{\text{makespan}_{opt}} \cdot 100\%.$$

$$\delta_{STS} = \frac{\text{makespan}_{STS} - \text{makespan}_{opt}}{\text{makespan}_{opt}} \cdot 100\%.$$

Таблица 1 средних и максимальных отклонений для каждого класса графов и каждого варианта буферизации:

Таблица 1: Средние и максимальные отклонения

Тип графа	Тип буферов	Ср. δ_{EFT} , %	Макс. δ_{EFT} , %	Ср. δ_{STS} , %	Макс. δ_{STS} , %
Слоистый	1 буфер	13.09	33.85	13.37	37.43
Слоистый	корневое	13.19	35.82	13.84	39.63
Случайный	1 буфер	9.51	32.48	9.56	39.42
Случайный	корневое	7.98	32.98	9.12	32.2

Здесь можно заметить, что алгоритмы работают по-разному в зависимости от типа буферов - на слоистых графах ЛП дает чуть меньший выигрыш относительно ЖА на графах с типом буфера 1, чем с типом буфера N_uniform. Для случайных графов ситуация противоположная.

Большие графы

Исследование масштабируемости

Поскольку ЛП работает значительно дольше ЖА, для исследования на больших графов ЛП не берется, следовательно необходимо сравнить между собой разные эвристики. Принципы сравнения эвристик ЖА заимствованы из работы [11]. При этом имеется возможность использовать треугольные графы с большим шагом размеров. На рисунке 15 видно, что разрывы во времени выполнения и в ЦФ двух ЖА могут быть заметными, тогда как на рисунке 16 видно, что алгоритмы практически совпадают, при этом значение ЦФ и время выполнения растет монотонно с увеличением размера графов. На

рисунке 17 видно, что эта монотонность может нарушаться, при этом время выполнения двух ЖА практически совпадают. В целом можно заметить, что везде ЖА с STS выполняется чуть быстрее и выдает расписание чуть лучшее по ЦФ, чем ЖА с EFT.

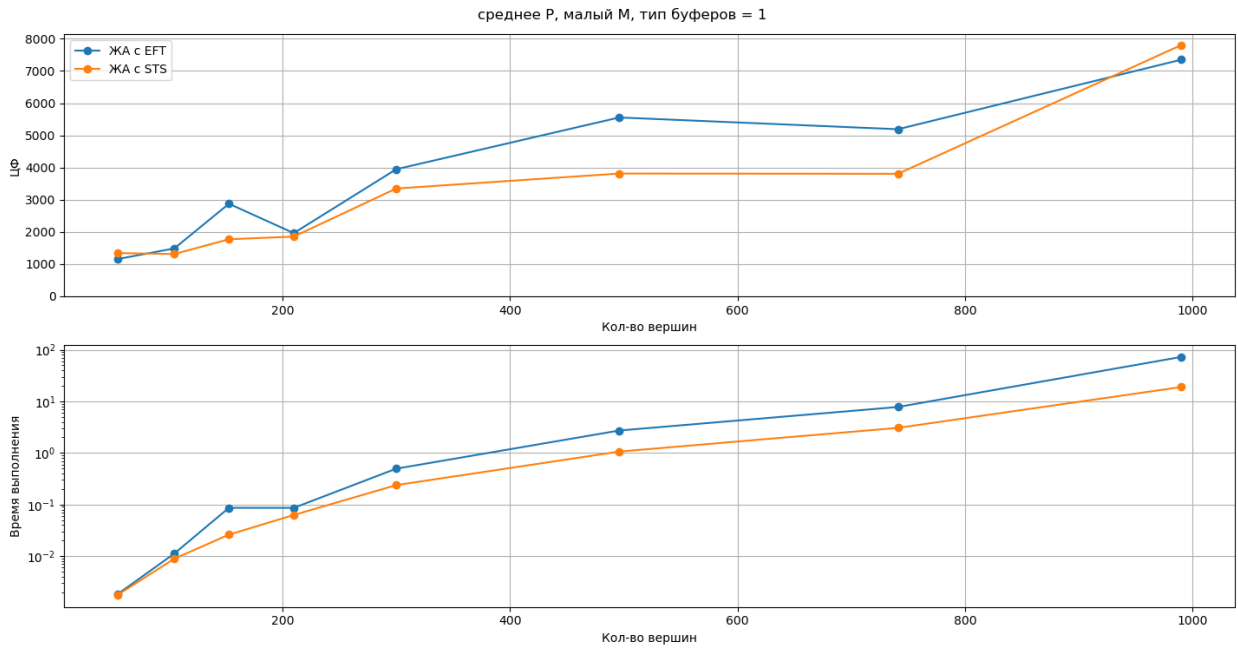


Рис. 15: Заметная разница между эвристиками

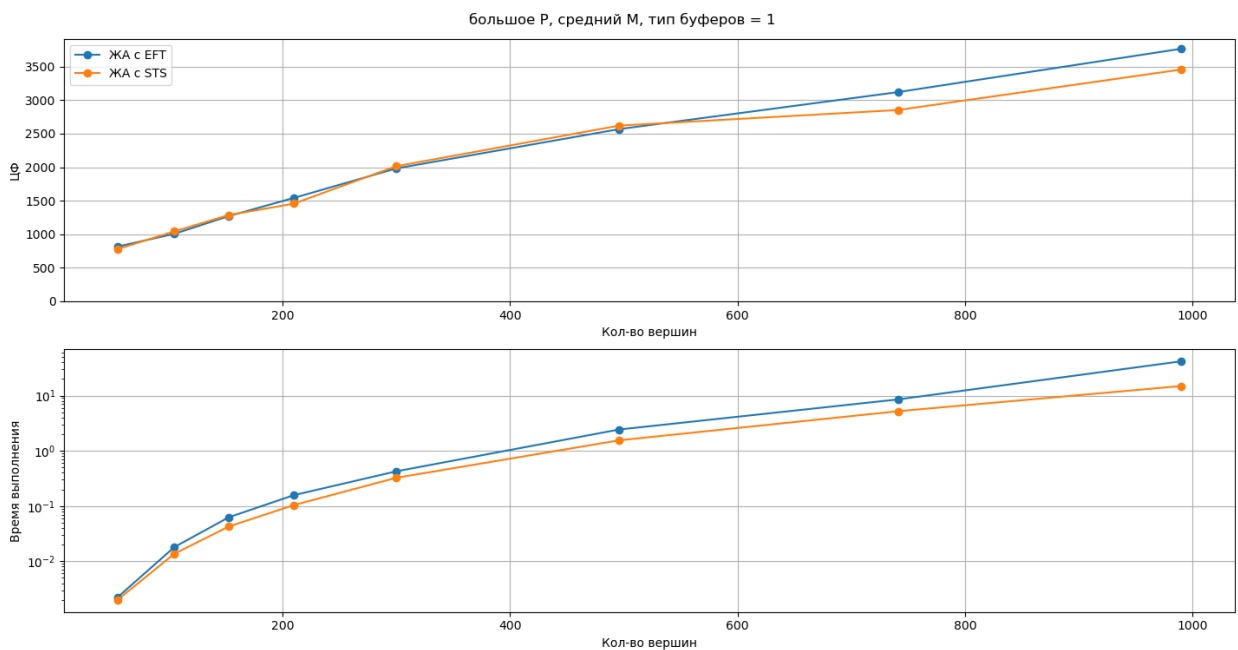


Рис. 16: Менее заметная разница

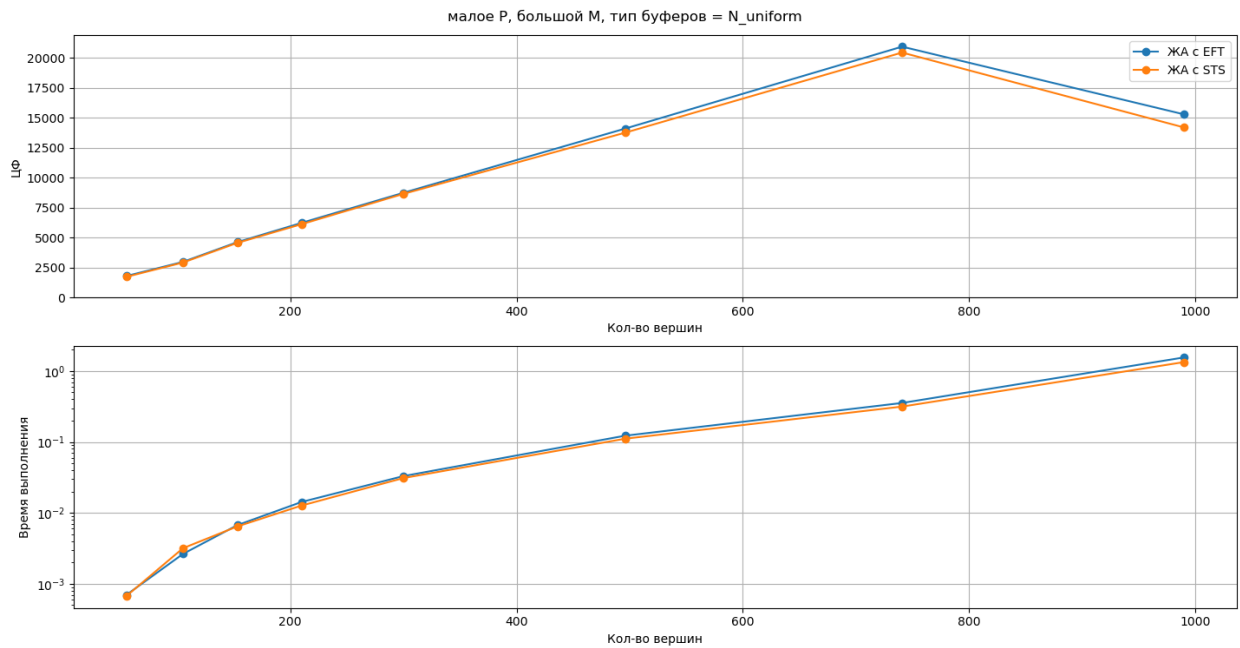


Рис. 17: Нарушение монотонности

Исследование качества решения

На рисунке 18 видно, как ЖА практически не различаются по ЦФ, есть случаи, когда одна эвристика чуть лучше другой и наоборот. При этом заметно общее снижение ЦФ при увеличении числа процессоров, так же как увеличение времени выполнения. На рисунке 19 видно, что на случайном графе размера 210 вершин ЖА с STS при малом числе процессоров заметно хуже ЖА с EFT, а на рисунке 20 заметна обратная ситуация, уже на треугольном графе размера 990 вершин.

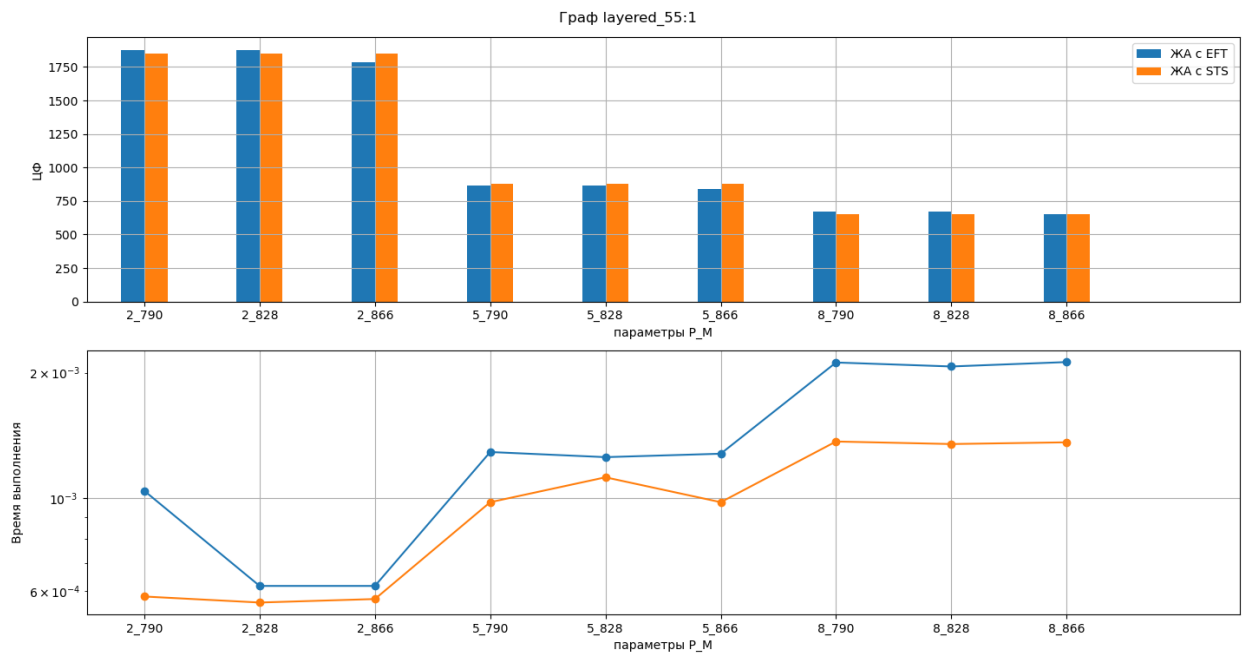


Рис. 18: Малые различия между алгоритмами

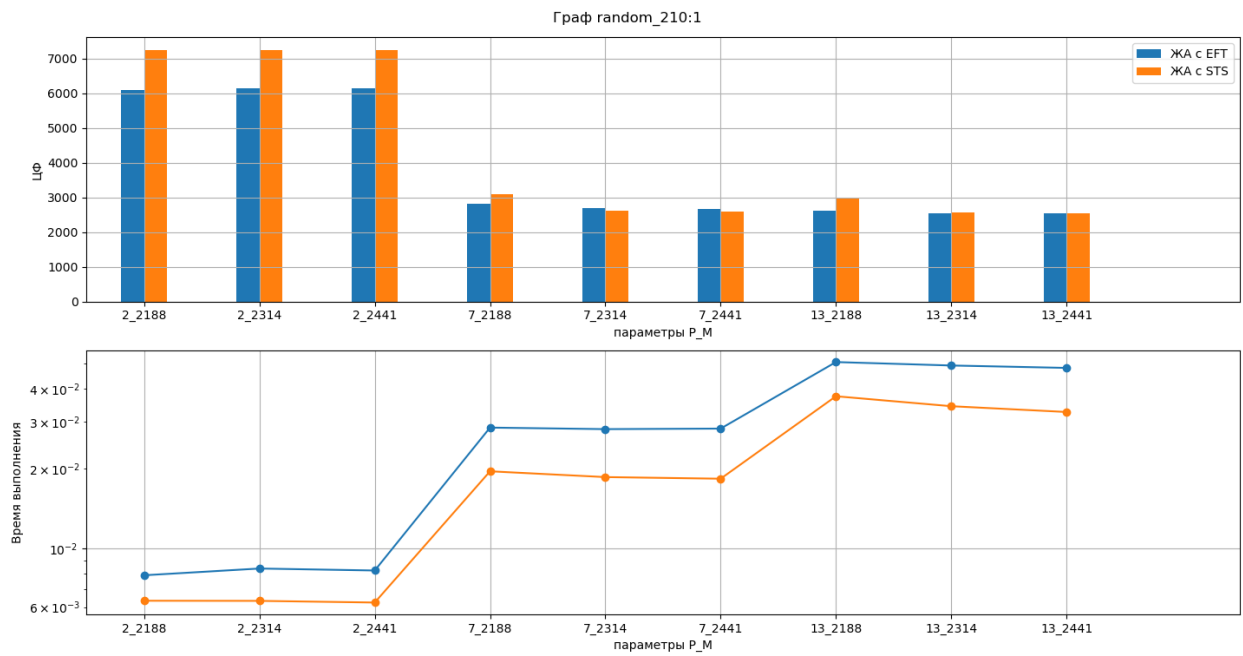


Рис. 19: Преимущество EFT

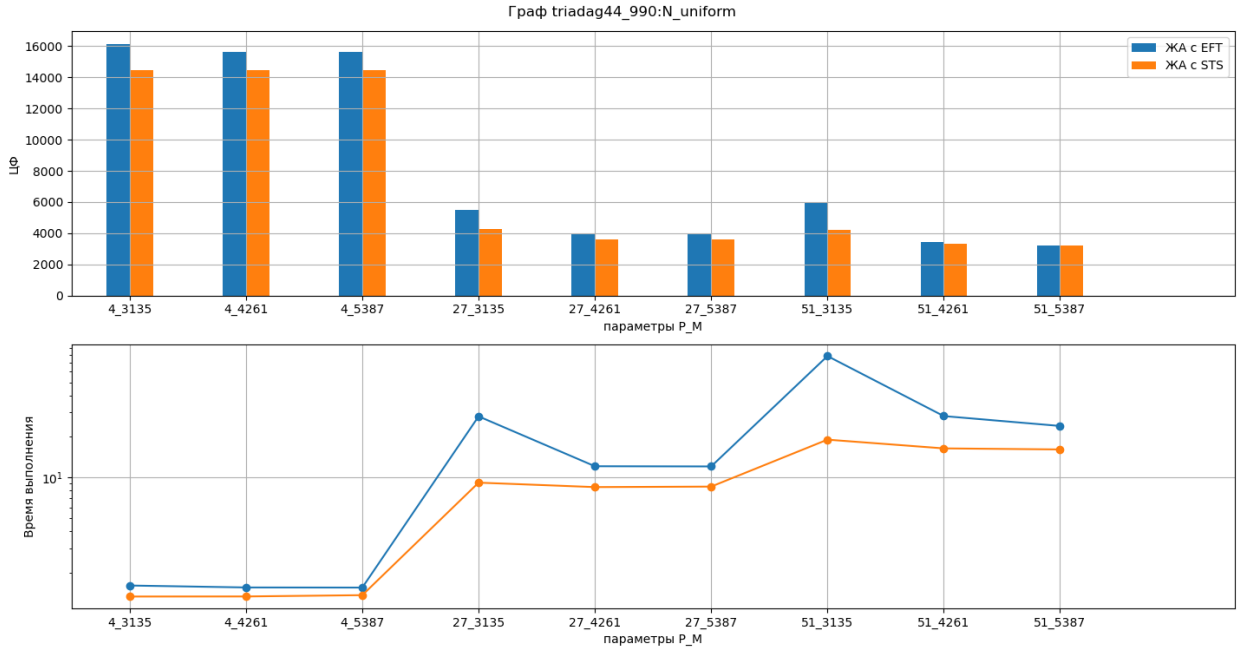


Рис. 20: Преимущество STS

Здесь уже не так заметно влияние увеличения лимита памяти на ЦФ и время работы, но заметно резкое снижение ЦФ при переходе от малого P к среднему. При этом время работы неизменно растет при увеличении числа процессоров.

Для обобщения результатов для каждого графа, набора (P, M) и варианта буферизации вычисляется относительное улучшение STS относительно EFT:

$$\Delta = \frac{\text{makespan}_{\text{EFT}} - \text{makespan}_{\text{STS}}}{\text{makespan}_{\text{EFT}}} \cdot 100\%.$$

Положительное значение означает, что STS лучше EFT.

Результаты обобщаются тепловой картой (рисунок 21) для фиксированного класса графов и типа буферов, по осям: размер графа и пары параметров P, M (малые, средние, большие) – цветом показывается Δ . На ней можно заметить, что для слоистых графов с типом буфера 1 есть параметры, на которых STS заметно лучше EFT, в основном это касается малого числа вершин. При большом числе вершин, а также при малом и среднем числе процессоров видно, что EFT лучше STS.



Рис. 21: Общее сравнение эвристик, белые поля - те параметры, на которых не завершился ЖА с STS

На рисунке 22 видно, что эвристика STS в абсолютном большинстве случаев хуже EFT на случайном графе с типом буферов 1, причем на более крупных графах при малых P и M разница доходит до 15-20%. На рисунке же 23 уже видно, как STS практически везде выигрывает у EFT, особенно на среднем и большом P и малом M, где разница доходит до 15-30%.

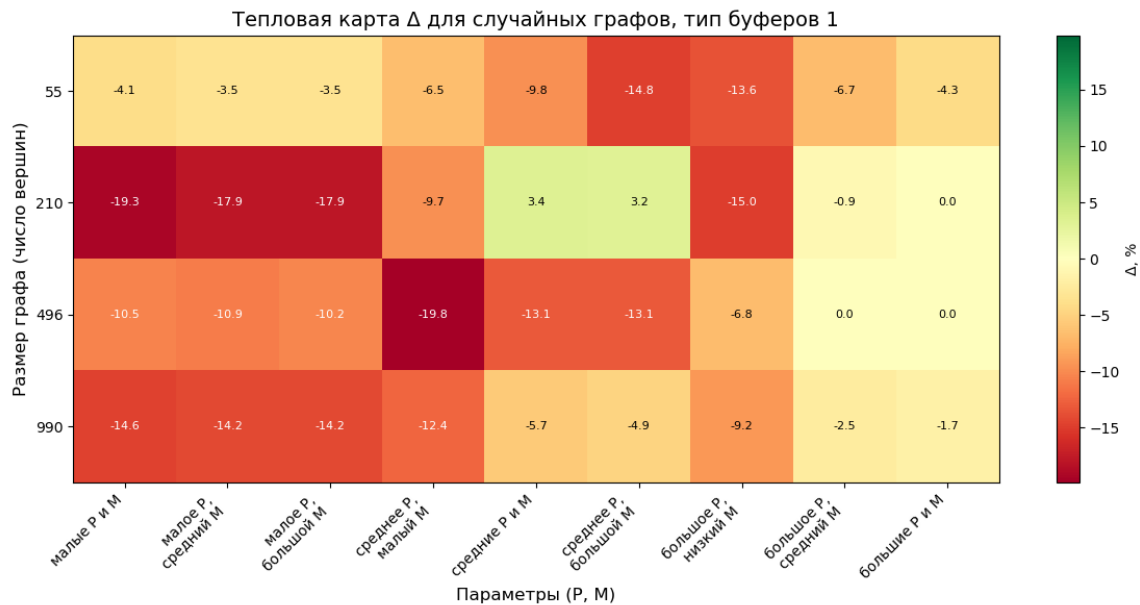


Рис. 22

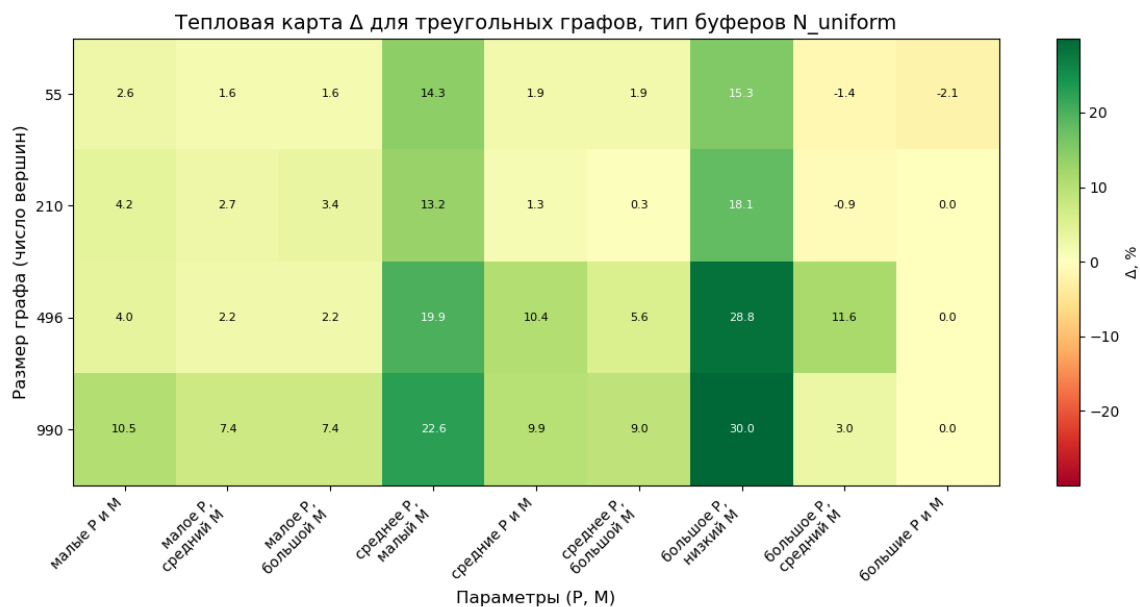


Рис. 23

5.5 Выводы

Проведённое экспериментальное исследование позволяет сделать следующие выводы.

Сравнение ЦЛП с жадными алгоритмами. На малых графах ($n \leq 21$) ЦЛП (SCIP) находит оптимальное расписание, однако уже при $n \geq 16$ время работы приближается к лимиту (3600 с) или превышает его. Решение ЦЛП всегда не хуже жадных, среднее отклонение δ_{EFT} составляет 9%–13%, максимальное — до 35%. Жадные алгоритмы работают на порядки быстрее, поэтому ЦЛП целесообразно применять только для получения эталонных решений на малых графах.

Сравнение эвристик EFT и STS. В целом эвристики не сильно различаются. На малых графах STS может быть чуть лучше на слоистых графах, но при увеличении размера графов EFT начинает показывать лучшие результаты. На случайных графах EFT почти всегда работает заметно лучше, тогда как на треугольных заметно лучше работает STS.

Влияние параметров P и M на качество и время работы. Увеличение числа процессоров P и лимита памяти M ожидаемо приводит к снижению makespan как для жадных алгоритмов, так и для оптимального расписания (ЦЛП). Наиболее заметное улучшение наблюдается при переходе от минимальных к средним значениям P и M ; дальнейший рост ресурсов даёт меньший относительный выигрыш. Для жадных алгоритмов снижение makespan при увеличении P и M не гарантируется, однако на практике оно происходит в подавляющем большинстве случаев. Время работы ЦЛП сильно зависит от P : при $P = 2$ решатель часто не укладывается в лимит даже на графах с $n = 18$, тогда как при больших P (4 и выше) успевает найти решение для всех малых графов. Для жадных алгоритмов время работы незначительно растёт с увеличением P и M , оставаясь на порядки меньше, чем у ЦЛП.

Рекомендации. Для графов более 20 вершин рекомендуется использовать жадные алгоритмы - нужную эвристику в зависимости от типа и размера графа, или запускать обе эвристики с выбором лучшего расписания. ЦЛП применим только для малых графов, когда требуется оптимальное решение.

6 Описание программной реализации

Все файлы доступны в репозитории¹ в ветке `multiprocessing`. Реализация ЖА содержится в папке `algorithms`, файлы по ЛП - в папке `LP`, по визуализации - в папке `visualisation`. Общий объем программ - около 221 Кб, около 5000 строк кода.

6.1 Реализация жадного алгоритма

Программная реализация жадных алгоритмов выполнена на языке C++ (стандарт C++17). В основе реализации лежит модульная архитектура, позволяющая легко добавлять новые эвристики и конфигурировать параметры (число процессоров, лимит памяти). Основные классы и их взаимосвязи показаны на диаграмме (рис. 24).

Базовый класс `BaseOptimization` Абстрактный класс, задающий интерфейс для всех алгоритмов оптимизации расписания. Содержит виртуальные методы `schedule()` (построение расписания), `setParams()/getParams()` (управление параметрами) и `copy()` (клонирование). Наследники должны реализовать конкретный метод `schedule_()`.

Класс `Greedy (EFT)` Реализует жадный алгоритм, использующий эвристику **Earliest Finish Time (EFT)**. Ключевая функция — `earliestStartOnProcessor2()` — просматривает интервалы занятости процессора и находит подходящее окно.

Класс `Greedy2 (STS)` Реализует эвристику **Smallest Time Space (STS)**, учитывающую как длительность работы, так и её влияние на будущую занятость памяти. Вычисление оценки вынесено в отдельные переменные `output_buffers_weight_sum` и `weighted_duration_sum` внутри основного цикла.

Вспомогательные структуры

- **ProcSlot** — описывает отрезок времени, занятый работой на процессоре (интервал $[start, finish)$).

¹URL: <https://github.com/Grenkas321/Coursework>

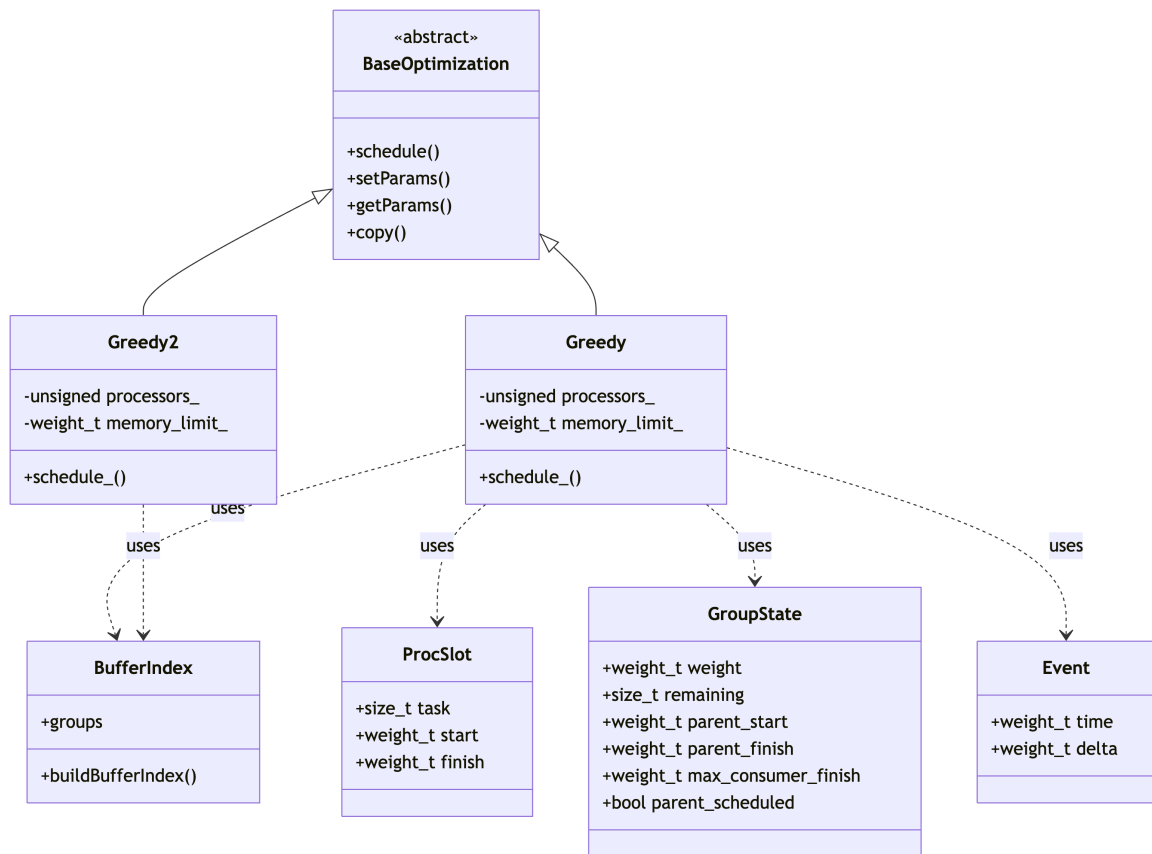


Рис. 24: Диаграмма классов жадного алгоритма

- **GroupState** — состояние буфера: объём памяти, число оставшихся потребителей, время старта/завершения родительской работы и максимальное время завершения среди уже назначенных потребителей.
- **Event** — событие изменения занятости памяти (выделение или освобождение), используется при проверке допустимости по памяти.
- **BufferIndex** — индекс буферов, построенный по входному графу: для каждой работы хранит список её буферов, а для каждого буфера — список потребителей и объём. Используется для инициализации состояний **GroupState**.

Учёт ограничения памяти Для каждой кандидатной работы проверяется, не превысит ли пиковое использование памяти заданный лимит M при помещении работы в предполагаемое время старта. Для этого:

1. Формируется список событий (выделение/освобождение) на основе уже зафиксированных буферов (поля `parent_scheduled`).
2. Добавляются события выделения для буферов самой кандидатной работы в момент её старта.
3. Для каждого родительского буфера, у которого кандидат является последним потребителем, добавляется событие освобождения в момент $\max(\text{finish}_{\text{parent}}, \text{finish}_{\text{consumer}}, \text{finish}_{\text{candidate}})$.
4. События сортируются по времени, вычисляется скользящая сумма занятой памяти; если в любой момент сумма превышает M , размещение отвергается.

Для графов с большим числом работ эта проверка является наиболее вычислительно затратной, поэтому она оптимизирована за счёт предварительного упорядочивания событий и раннего выхода при превышении лимита.

Особенности реализации

- Все времена и объёмы памяти представлены целочисленным типом `weight_t` (псевдоним `long long`).
- Процессоры нумеруются с 0 до $P - 1$, расписание на каждом процессоре хранится как отсортированный вектор интервалов `ProcSlot`.
- Множество готовых работ поддерживается в `std::set` (автоматическая сортировка по идентификатору).
- Для ускорения поиска стартового окна на процессоре используется линейный проход по интервалам — в силу того, что число интервалов на процессор не превышает числа работ, а общее число работ редко превышает несколько тысяч, этого достаточно.

6.2 Реализация подхода на базе ЦЛП

Подход, описанный в разделе 4, реализован в виде набора скриптов на языке Python и Bash. Основные компоненты:

- `make_input_times_step2_2.py` – транслятор, преобразующий входной граф работ в описание задачи ЦЛП на входном языке решателя SCIP (файл с расширением `.lp`). Для каждого графа генерируется 10 упорядочений (см. подраздел 5.4) Структура `.lp`-файла соответствует распределению, приведённому в табл. 2.
- `run_many_make_inputs_time_for_LP.py` – автоматизирует вызов транслятора для всех входных графов из каталога экспериментов. Из имени каждого файла извлекаются значения числа процессоров P и лимита памяти M (см. подраздел 5.3), и эти параметры подставляются в глобальные переменные скрипта `make_input_times_step2_2.py`.
- `run_many_solvers.py` – управляет выполнением серии экспериментов через обёрточный скрипт `run_scip2.sh`. Ограничение времени работы решателя задаётся в файле `only_time.set` (параметр `limits/time = 3600`).
- `run_scip2.sh` – реализует параллельный запуск 8 экземпляров SCIP. Логи работы сохраняются в отдельные каталоги.
- `file_reader_and_making_json.py` – извлекает из логов SCIP значения переменных:
 - s_i (время старта работы i),
 - p_{ri} (номер процессора, назначенного работе i),
 - вычисляет `makespan` как $\max_i(s_i + d_i)$.

Результат записывается в JSON-файл, полностью совместимый по структуре с выходными файлами жадных алгоритмов.

Для генерации входных графов и подготовки наборов экспериментов использованы дополнительные скрипты:

Таблица 2: Распределение переменных и ограничений по секциям .lp-файла

Секция	Переменные / ограничения	Пояснение
Integer	$l, s_i, \text{start}_{i,j}, \text{max_end}_{i,j}$	Целевая переменная, времена старта работ, начала и макс. завершения буферов
Binary	$m_{ij}, p_{ri}, t_{ij}, z_{ij}^{pq}, z_{ij}^{pq}, a_{ij}^{pq}, aa_{ij}^{pq}$	Отношения порядка, назначение на процессоры, пересечения буферов
Subject to	(1)–(2), (7), (10)–(11), (12)–(15) и др.	Основные линейные ограничения
Bounds	$m_{ii} = 1, m_{ij} = 1$ для $(i, j) \in E$ (включая транзитивные)	Фиксированные значения переменных предшествования
Minimize	l	Целевая функция

- `layered_generator.py`, `random_generator.py`, `triadag_generator.py` – создают графы в «старом» формате (вершина, вес, список потомков) без учёта буферов.
- `converter.py` – преобразует «старые» графы в новый формат с буферами: назначает случайные длительности работ ($\in [10, 100]$) и веса буферов ($\in [1, 100]$), формирует два варианта группировки рёбер в буферы – один буфер (`_1_`) и равномерное корневое распределение (`_N_uniform_`).
- `finding_M_and_P.py` – для каждого полученного графа вычисляет на практике максимальное используемое число процессоров $P_{\max}$ и максимальную пиковую память $M_{\max}$ (при заведомо избыточных ресурсах). Затем создаёт 9 вариантов входных файлов (3 значения $P \times 3$ значения M) согласно сетке, описанной в подразделе 5.3.

Скрипты `run_scip2.sh` и `only_time.set` обеспечивают воспроизводимость экспериментов и возможность ограничения времени счёта. Все сгенерированные расписания (в

формате JSON) и логи сохраняются в структурированном виде, что позволяет автоматизировать последующий анализ результатов.

6.3 Визуализатор временной диаграммы расписания

Для наглядного представления построенных расписаний (как полученных жадными алгоритмами, так и с помощью ЦЛП) разработан скрипт `gantt_and_memory.py` на языке Python с использованием библиотеки `matplotlib`. Он создаёт совмещённое изображение, состоящее из двух диаграмм:

1. **Диаграмма Ганта** – отображает загрузку процессоров во времени. Каждая работа изображается цветным прямоугольником, внутри которого указан её идентификатор. По горизонтальной оси отложено время, по вертикальной – номера процессоров.
2. **График использования памяти** – показывает, как меняется суммарный объём занятой памяти в процессе выполнения расписания. По вертикальной оси отложен объём памяти, по горизонтальной – время. Дополнительно красной горизонтальной линией отмечается заданный лимит памяти M .

Входные данные:

- JSON-файл с расписанием, содержащий для каждой работы номер процессора, время старта и завершения.
- Исходный TXT-файл графа работ в новом формате (с буферами), необходимый для определения, какие буферы активны в каждый момент времени.

Ключевые компоненты реализации:

- Функции рисования стрелок (`draw_horiz_arrow`, `draw_vertical_arrow`, `draw_arc_arrow`, `draw_loop_arrow` и др.) – используют `FancyArrowPatch` из `matplotlib.patches`. Они проводят дуги и линии между работами, визуализируя зависимости по данным (рёбра графа). Тип стрелки зависит от взаимного расположения работ: горизонтальная, вертикальная, диагональная или дугообразная.

- Класс `ResourceVisualizer` – накапливает события изменения памяти (выделение и освобождение буферов) для каждого момента времени. Метод `visualize()` строит ступенчатую диаграмму, где каждый прямоугольник соответствует периоду активности конкретного буфера. Метод `add_horizontal_line()` добавляет линию заданного лимита памяти.
- Функция `get_task_coordinates` – вычисляет координаты работы на диаграмме Ганта по её времени старта, длительности и номеру процессора.

Выходные данные: Скрипт отображает полученное изображение на экране (при интерактивном запуске) и может сохранить его в файл (например, в формате PNG) с высоким разрешением (300 dpi).

Визуализатор активно используется при отладке алгоритмов, а также для качественного анализа полученных расписаний в ходе экспериментального исследования. С его помощью были построены, например, рисунки 2, 3, 13, 14.

6.4 Структура входного и выходного файлов

Входной файл (описание графа работ)

Входной файл имеет расширение `.txt` и содержит строки в кодировке UTF-8. Первая строка является заголовком и содержит названия полей:

```
id time weight_1: child_1 ... child_n, weight_2: child_1 ... child_n, ...
```

Каждая последующая строка описывает одну работу (вершину графа) и имеет следующий формат:

```
<id> <time> <буфер_1>, <буфер_2>, ..., <буфер_k>
```

где

- `<id>` — целочисленный идентификатор работы (от 0 до $n - 1$);
- `<time>` — целочисленная длительность выполнения работы;
- `<буфер_i>` имеет вид `<вес>: <список потомков>`, где `<вес>` — целое число, задающее объём памяти, занимаемый данным буфером, а `<список потомков>` —

разделённые пробелами идентификаторы работ-потребителей этого буфера. Если у работы нет исходящих рёбер, в строке указывается единственный буфер 0: (нулевой вес, пустой список потомков).

Буферы в строке разделяются запятой с пробелом (‘, ‘). Внутри одного буфера после двоеточия может быть перечислено несколько потомков, что означает, что все они используют один и тот же буфер (и, следовательно, разделяют его данные). Пример строки для работы с двумя буферами:

```
2    36    10: 3 6 7, 73: 4 8
```

Это означает, что работа 2 длится 36 единиц времени, создаёт два буфера: первый весом 10 используется работами 3, 6, 7; второй весом 73 используется работами 4 и 8.

Выходной файл (расписание)

Результат работы жадного алгоритма или решателя ЦЛП сохраняется в файл формата JSON с расширением `.json`. Файл содержит один объект, ключом которого является имя входного графа (без расширения), а значением — объект с полями, описывающими расписание. Структура JSON-объекта:

```
{
  "имя_графа": {
    "makespan": целое,
    "runtime_sec": число с плавающей точкой,
    "size": целое,
    "schedule": {
      "id_вершины": {
        "processor": целое,
        "start": целое,
        "finish": целое
      },
      ...
    }
  }
}
```

Поля означают:

- **makespan** — общая длительность расписания (максимальное время завершения среди всех работ);
- **runtime_sec** — реальное время работы алгоритма в секундах;
- **size** — число вершин в графе;
- **schedule** — словарь, где каждому идентификатору вершины соответствует словарь с тремя полями:
 - **processor** — номер процессора (начиная с 0), на котором выполняется работа;
 - **start** — время начала выполнения;
 - **finish** — время завершения (**start** + длительность работы).

Заключение

В результате выполнения работы получены следующие основные результаты:

1. Разработан жадный алгоритм списочного планирования для многопроцессорной системы с ограничением на суммарный объём памяти. Реализованы две эвристики выбора работы: EFT (минимизация времени завершения) и STS (учёт «затрат» на память).
2. Выполнено сведение задачи к задаче целочисленного линейного программирования (ЦЛП) с использованием переменных, моделирующих пересечение интервалов активности буферов. Входные графы транслируются на язык .lp решателя SCIP.
3. Проведено экспериментальное исследование на трёх классах графов (слоистые, случайные, треугольные) размером от 10 до 990 вершин. Показано, что:
 - ЦЛП находит оптимальное решение для всех малых графов ($n \leq 21$), однако при $n \geq 16$ и $P = 2$ время работы приближается к лимиту 3600 с; для больших графов ЦЛП не применим.
 - Жадные алгоритмы работают на порядки быстрее (миллисекунды–секунды). Среднее отклонение makespan от оптимума составляет 9–13%, максимальное – до 35%.
 - Сравнение эвристик EFT и STS показало, что STS лучше на треугольных графах и при больших ресурсах (P, M), но уступает EFT на случайных графах с малым числом процессоров. Рекомендуется запускать обе эвристики и выбирать лучшее расписание.

Направления дальнейших исследований включают:

- поддержку целевых систем с процессорами различной производительности (длительность работы зависит от процессора);
- применение методов машинного обучения для выбора эвристики или предсказания «хороших» упорядочений переменных для ЦЛП;
- разработку гибридных алгоритмов, комбинирующих жадные эвристики и точные методы для графов среднего размера (50–200 вершин).

Список литературы

- [1] А. П. Ершов. Сведение задачи распределения памяти при составлении программ к задаче раскраски вершин графов. *Доклады Академии наук СССР*, 142(4), 1962.
- [2] V. Prakash, S. Bawa, and L. Garg “Multi-Dependency and Time Based Resource Scheduling Algorithm for Scientific Applications in Cloud Computing,” *Electronics*, vol. 10, p. 1320, 2021.
- [3] И. П. Савицкий. Жадные алгоритмы для построения многопроцессорного списочного расписания. Выпускная квалификационная работа, Московский Государственный Университет, Москва, 2023.
- [4] А. К. Сенюшкина. Подход на основе линейного программирования к минимизации пикового использования ресурса в однопроцессорных системах. Выпускная квалификационная работа, Московский Государственный Университет, Москва, 2025.
- [5] Y. Wang, D. Liu, Z. Qin, Z. Shao. Memory-aware optimal scheduling with communication overhead minimization for streaming applications on chip multiprocessors. In *2010 31st IEEE Real-Time Systems Symposium (RTSS)*, pages 41–50, San Diego, CA, USA, 2010. DOI: 10.1109/RTSS.2010.16.
- [6] J. Herrmann, L. Marchal, Y. Robert. Memory-aware list scheduling for hybrid platforms. Research Report n° 8461, INRIA, February 2014.
- [7] S. Venugopalan, O. Sinnen. Optimal linear programming solutions for multiprocessor scheduling with communication delays. In *Proc. Algorithms and Architectures for Parallel Processing*, pages 129–138, 2012.
- [8] P. A. Papp, T. Böhnlein, A.-J. N. Yzelman. Multiprocessor scheduling with memory constraints: fundamental properties and finding optimal solutions. *arXiv preprint arXiv:2507.17411*, 2025.
- [9] V. V. Balashov, V. A. Kostenko, I. A. Fedorenko, I. P. Savitsky, C. Sun, J. Gao, L. Zhou, J. Sun. Hybrid algorithm for multiprocessor scheduling with makespan minimization and constraint on interprocessor data exchange. In *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*, Rome, Italy, 2023.

- [10] В. Н. Шевченко, Н. Ю. Золотых. *Линейное и целочисленное линейное программирование*. Издательство Нижегородского госуниверситета им. Н.И. Лобачевского, Нижний Новгород, 2004.
- [11] В. А. Крет. Построение многопроцессорных расписаний с ограничением на долю межпроцессорных передач на основе жадных стратегий. Выпускная квалификационная работа, Московский Государственный Университет, Москва, 2025.

Приложение

На рисунке 25 видно, что уже на 14-16 вершинах при малом Р ЛП работает близко к 1000 секундам, а начиная с 17 вершин и вовсе не завершается. На рисунке 26 же видно, что при большом Р ЛП завершается на графах вплоть до 20 вершин, при этом часто оказываясь значительно лучше обоих ЖА по makespan и ни разу им не уступая.

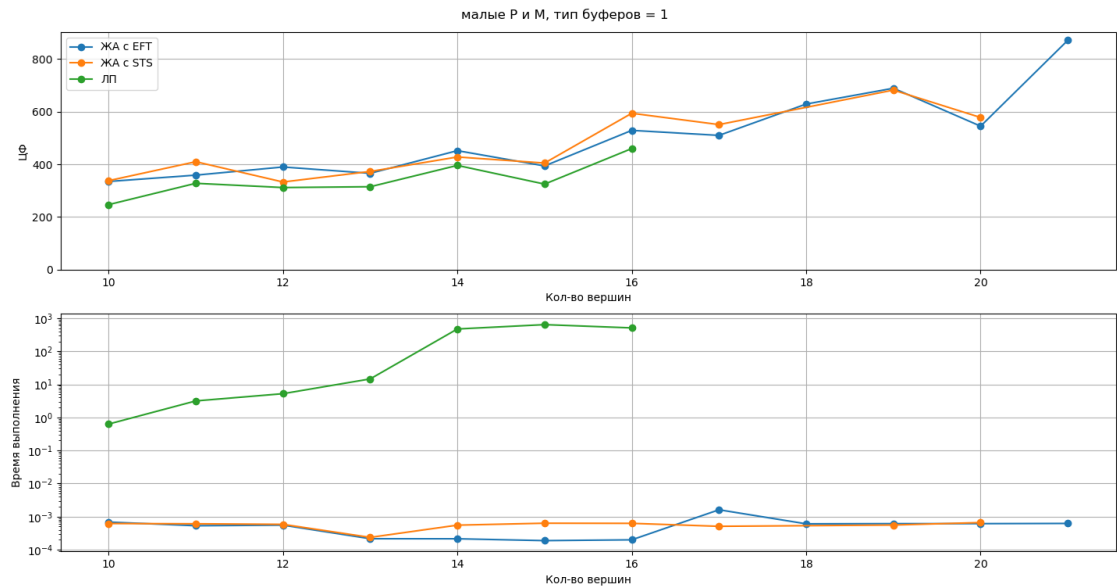


Рис. 25

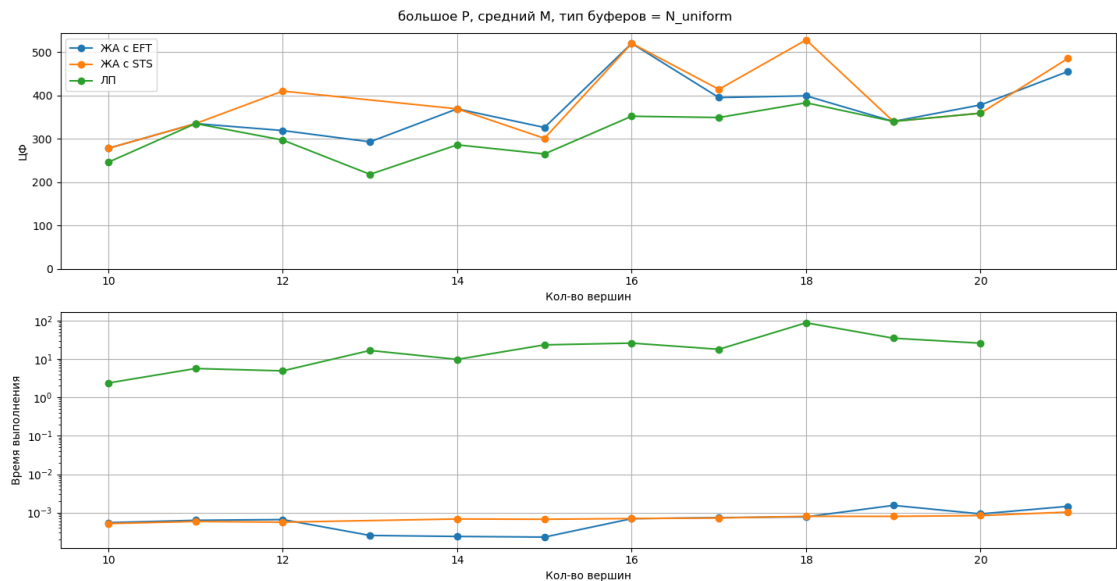


Рис. 26

На рисунке 27 видно, что ЖА с STS не всегда завершается, в данном случае ему при любом числе процессоров не хватает минимального лимита памяти. На рисунке 28 можно наблюдать, как плавно снижается значение ЦФ всех алгоритмов при увеличении доступного числа процессоров и памяти, при этом также можно заметить, что на случайном графе ЛП работает заметно быстрее, чем на предыдущем слоистом. На рисунке 29 видно, как на графе с 15 вершинами уже долго работает ЛП при 2 доступных процессорах, но при этом всегда находит заметно лучшее расписание, так же как при малом лимите памяти. На рисунке 30 уже видно, что при 16 вершинах ЛП не завершается при $P = 2$. И на рисунке 31 видно, как соотносятся алгоритмы на слоистом графе большого размера, видно, что там, где ЛП завершается, он часто дает заметно лучшее расписание по сравнению с ЖА.

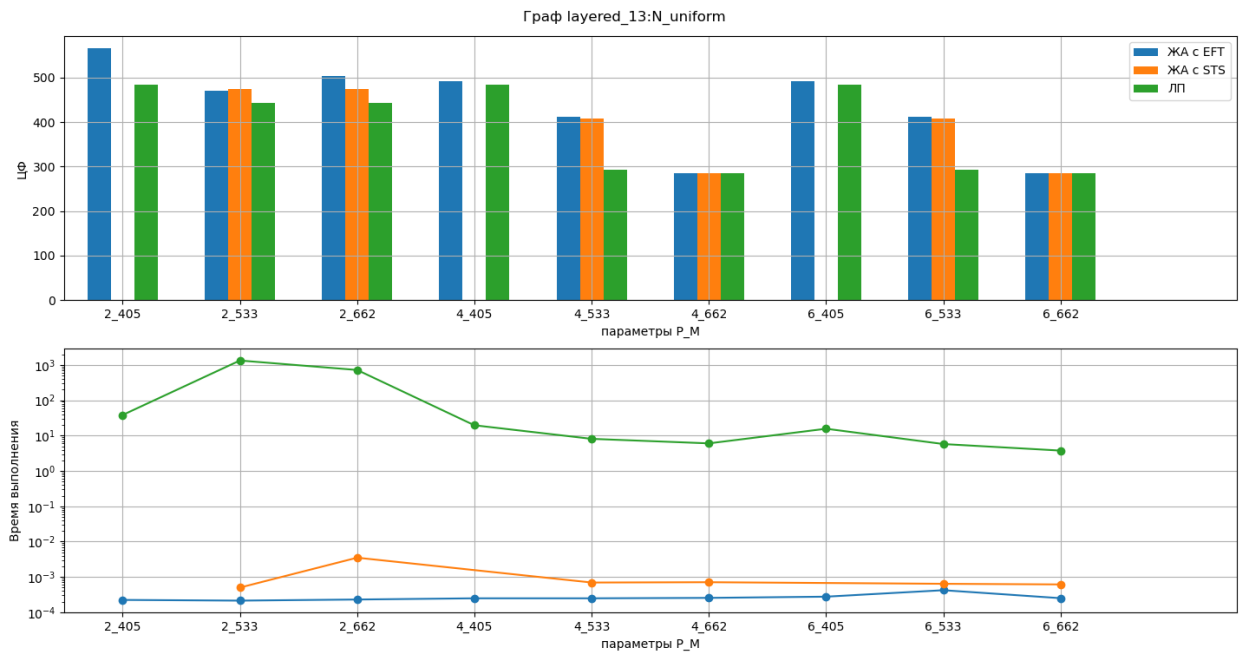


Рис. 27

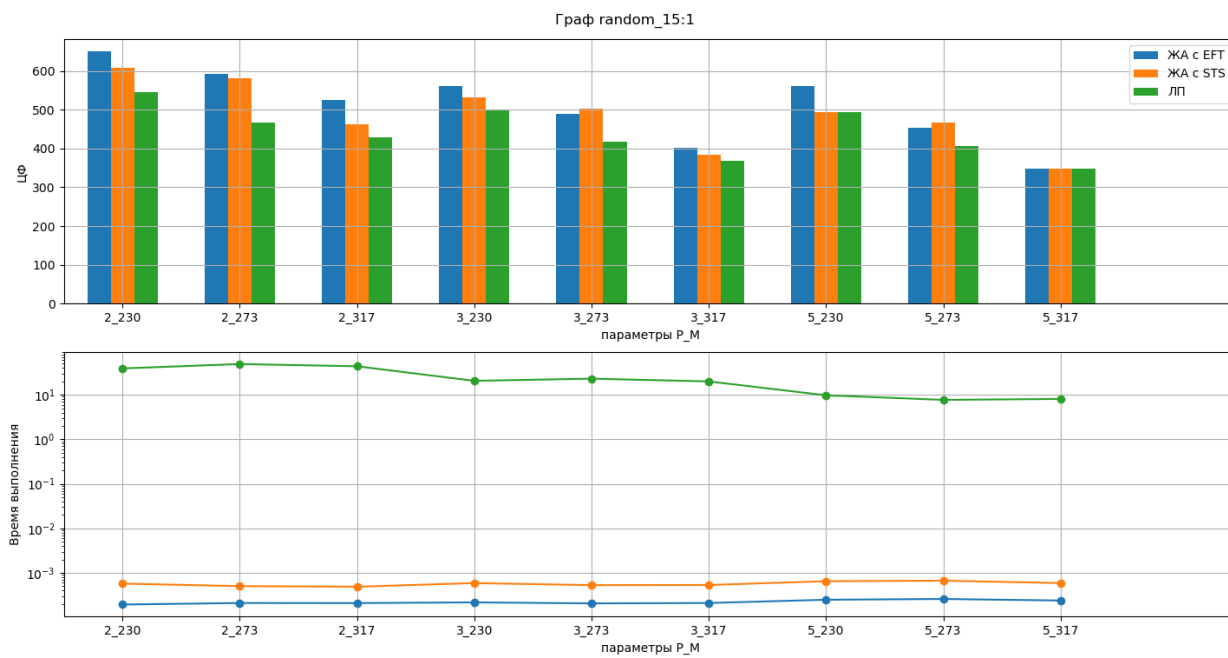


Рис. 28

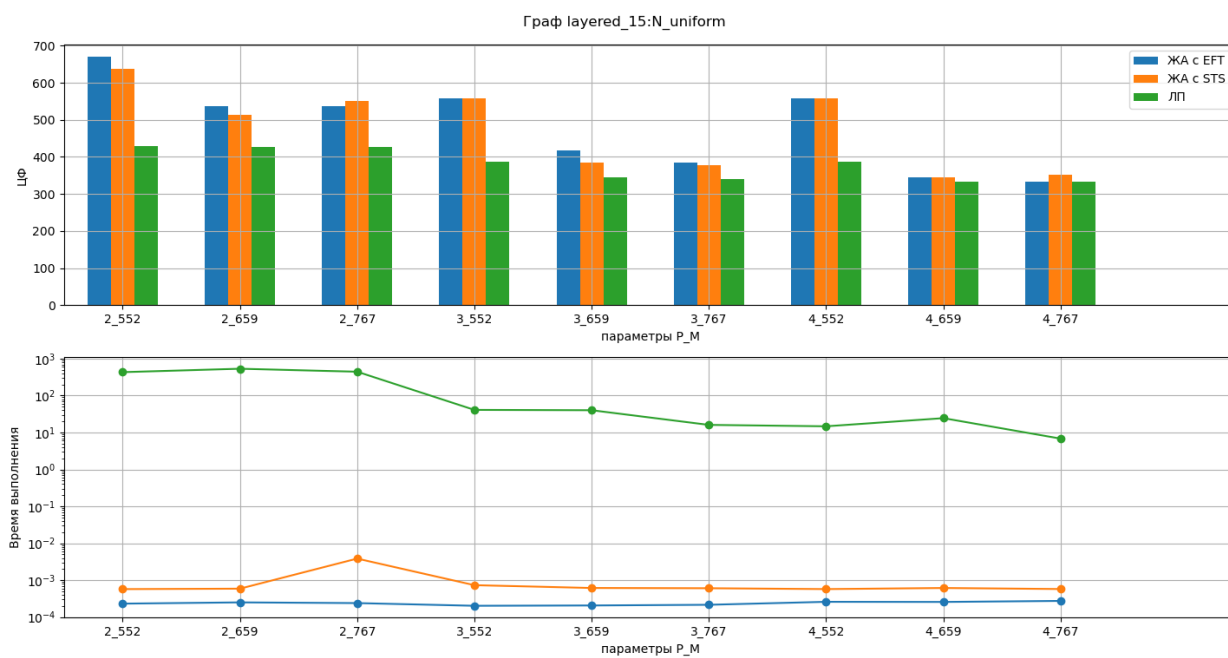


Рис. 29

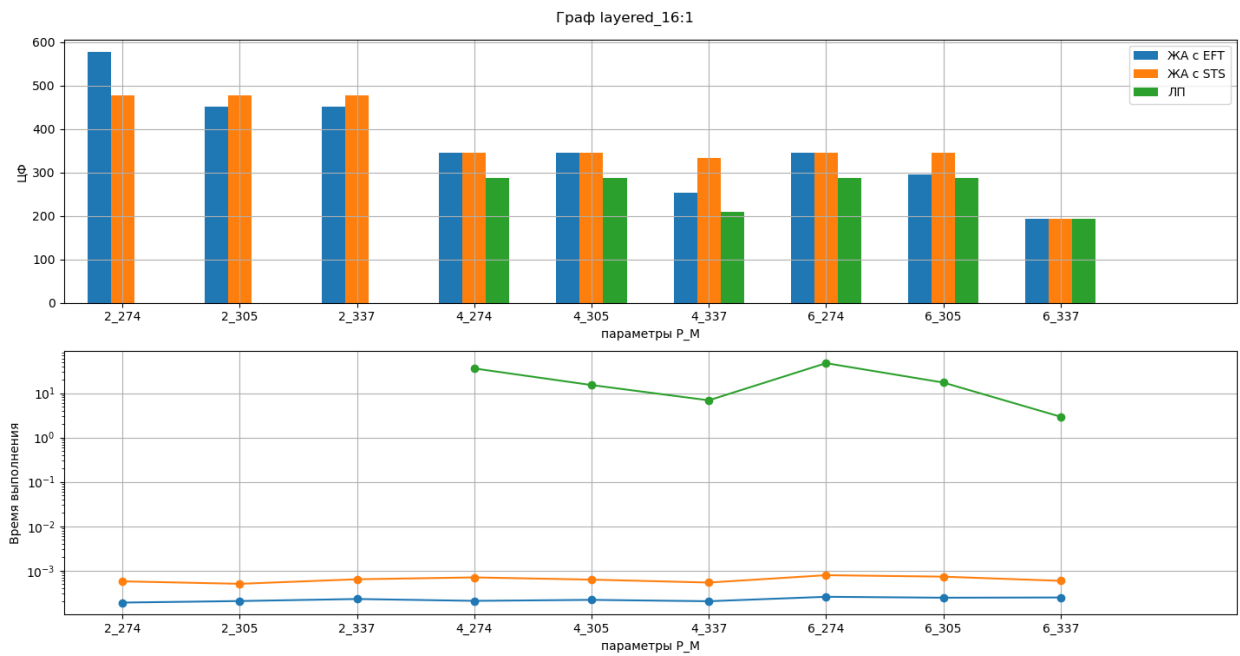


Рис. 30

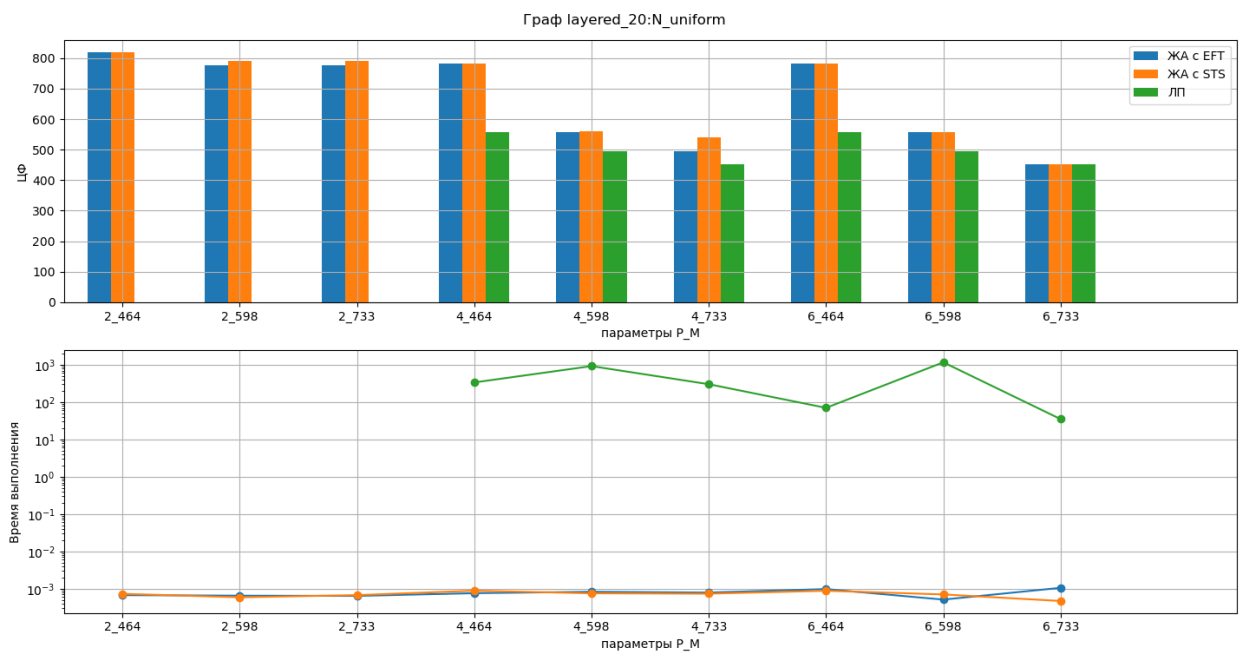


Рис. 31

На рисунках 32- 33 видно, как ЖА с EFT может быть заметно лучше ЖА с STS, граф - layered_12_1_2_321. На рисунках 34- 35 видно обратную ситуацию, граф - layered_16_1_2_274.

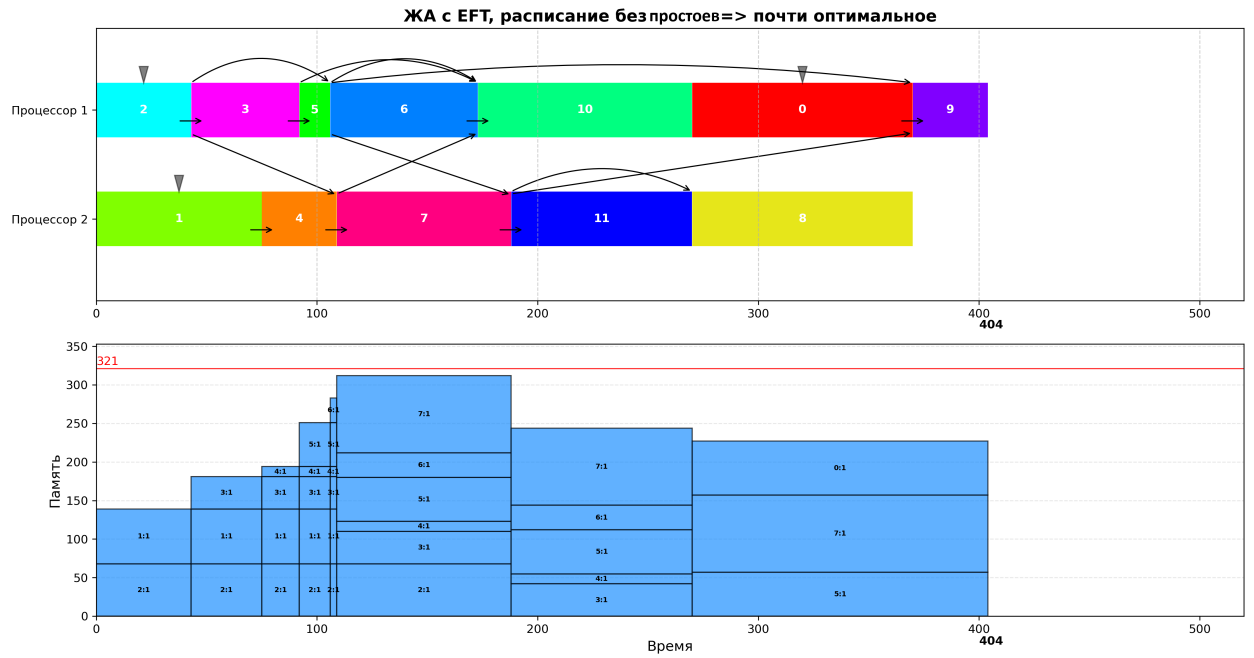


Рис. 32

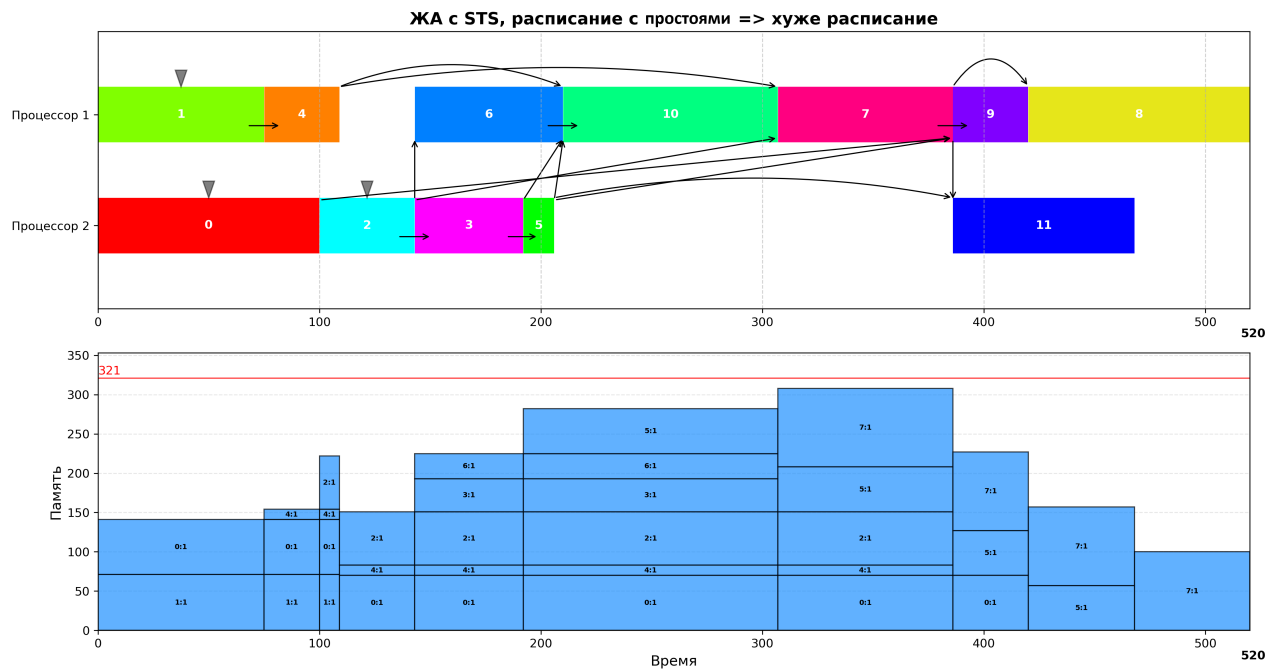


Рис. 33

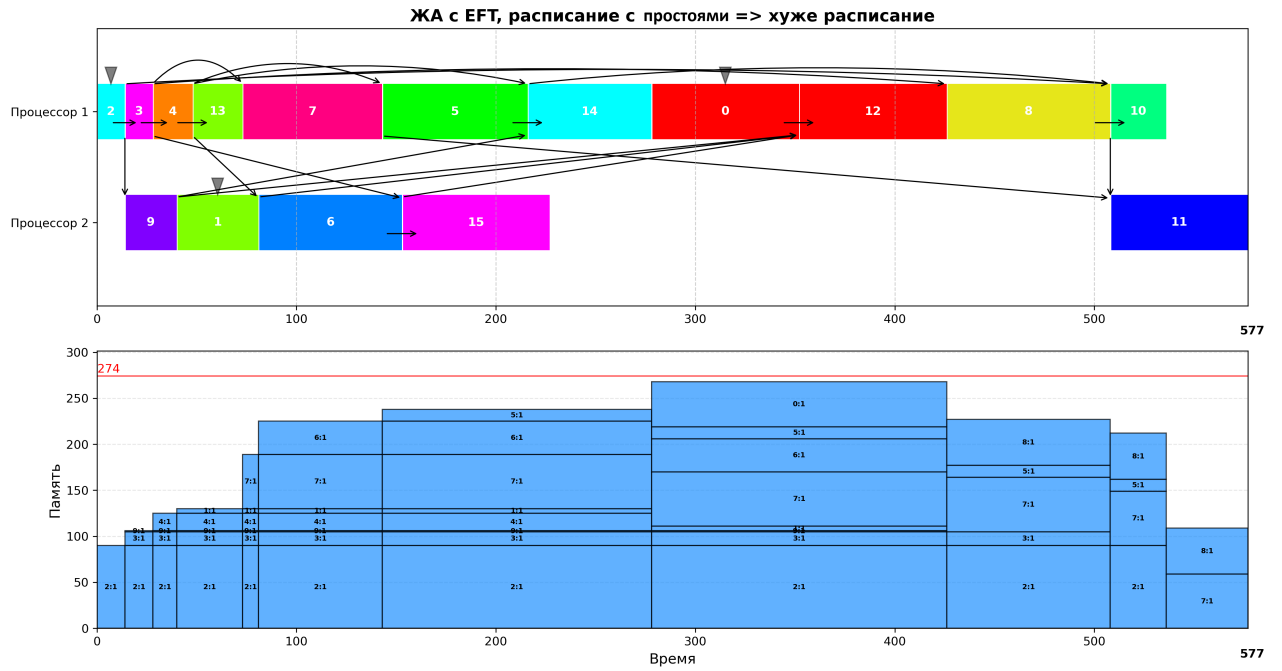


Рис. 34

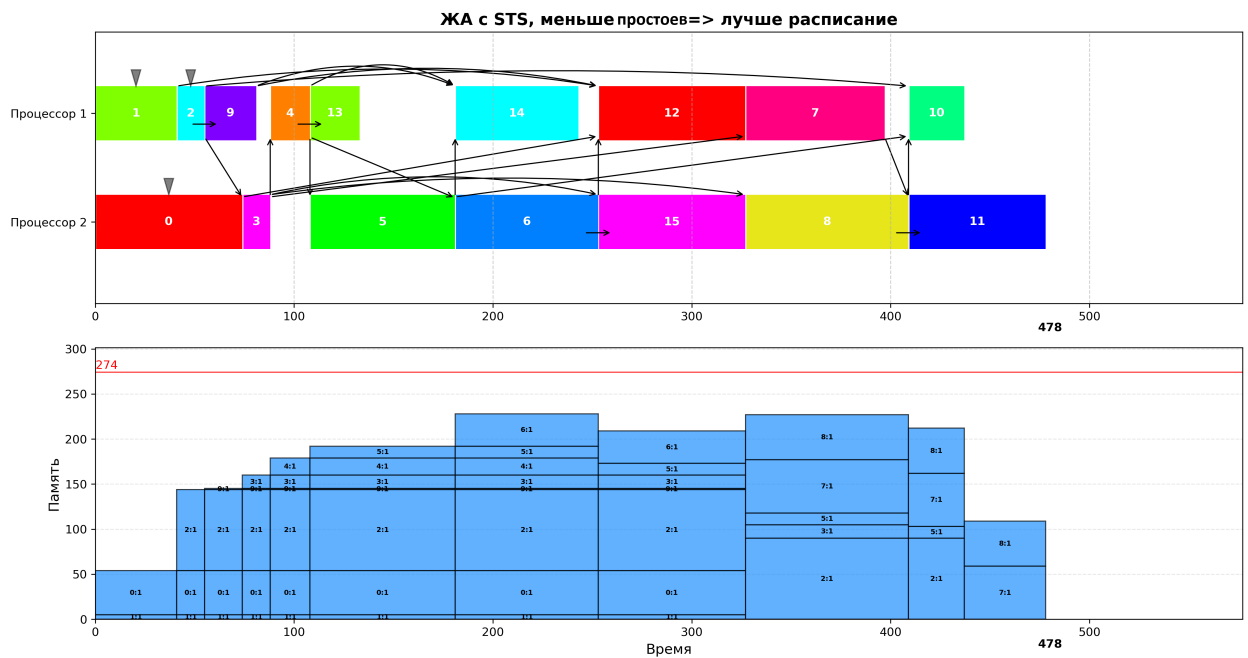


Рис. 35